# Vorkurs Mathematik
-
# Introductory Course Mathematics
## Part 1 (discrete, non-continuous)

Michael Luttenberger

Technische Universität München
School of Computation, Information and Technology
Department of Computer Science

2026/09/07

For personal use only, no commercial use – TUM CIT Department of Computer Science

1 Basic concepts



For personal use only, no commercial use – TUM CIT Department of Computer Science

1 Basic concepts

Basic concepts of logic

Sets

Sequences, tuples, and words

The natural numbers, proof by induction

Relations

Functions



For personal use only, no commercial use – TUM CIT Department of Computer Science

# Ambiguity

- ⚠ Central to logic and mathematics is the goal to be unambiguous (well-defined, well-formed, well-posed, well-founded), i.e. there is just one unique way of understanding (interpreting) a statement (proposition, assertion, declarative sentence):
  - "I saw a bat."

    Ambiguous semantics: how to read/interpret "saw" and "bat"?
  - "I saw a man on the hill with a telescope"

    Ambiguous syntax: how to group the words? E.g.:

    "(I saw (a man (on the hill) (with a telescope)))" or

    "(I saw (a man (on the hill)) (with a telescope))" or

    "(I saw (a man (on the hill (with a telescope))))"
- Brackets (also: parentheses) "$(, ), [, ], \{, \}, \langle, \rangle$" (and also punctation marks) are used to group words in order to avoid ambiguous syntax. (See Dyck word, well-formed string.)
- Definitions are used to associate a unique meaning/interpretation with a word (a "term") in a certain context (area of research, book, article, debate).

  (definire: "to bound, limit, explain"; word: "smallest discrete, meaningful unit of language")

For personal use only, no commercial use – TUM CIT Department of Computer Science

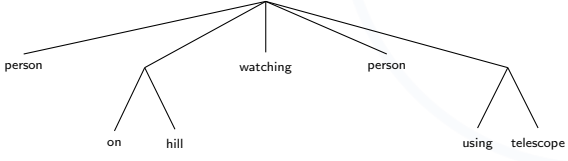
"Black-white simple comic style: one person on a hill watching another person using a telescope"

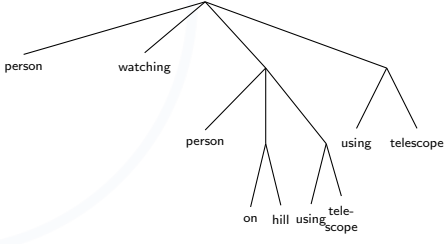


(person,(on,hill),watching,person,(using,telescope))



(person,watching,(person,(on,hill),(using,telescope)),(using,telescope))




Natural language is ambiguous, there is no unique meaning in general. Even a machine must guess what you mean if you are imprecise. We need a "well-defined fragment" of a language.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Propositions



- Mathematics builds up on logic ("logikè téknē": art/study of correct reasoning).
- In logic "proposition" refers to the actual content of a declarative sentence to which we can either agree (true) or disagree (false) (only two truth values).

  (Outside of logic, "proposition" can also mean "sentence" (see e.g. here (5,6,7,8)); also, "statement", "assertion"', "claim" can have both meanings (e.g. here).)

⚠ A proposition can have several **syn**onymous formulations (1) (the form we give a proposition: concrete sentence (5,6), formula (1,8), code (7), expression (2), ...).

The truth value (true, false) of a proposition can depend on the situation (context, interpretation, "world", case, "perspective", memory contents, book, author, ...).

EG "It is raining" vs. "Rain is falling".

"$0$ is a natural number" vs "$0$ is a nonnegative integer".

"let $x := 2$; $x > 0$" vs. "let $x := -2$; $x > 0$" (meaning of $x$ defined in context.)

- Central questions of logic: satisfiability and validity

  A satisfiable proposition is true in at least one (hypothetical) situation (else "unsatisfiable"). A valid proposition is true in all situations (else "invalid").

  ("validus": "strong, well, healthy", "withstands all counterarguments".)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Bandersnatches, connectives, quantifiers

- To decide whether a statement is satisfiable or even valid, we need to agree on the meaning of certain words (well-defined semantics).

EG "If all bandersnatches are borogoves, and all borogoves are slithy, then all bandersnatches are slithy" (Prof. Espara(n)za)

- The words "bandersnatches", "borogoves", "slithy" are taken from the poem Jabberwocky by Lewis Caroll.

  At the time of writing, these words had no meaning associated with them (no entry in a dictionary) (by now there is a black mirror episode).

- Still, wrt. the standard definitions how to read connectives and quantifiers in logics and mathematics, above sentence is valid as we will see.

  (I.e. the concrete interpretation of "bandersnatch" does not matter.)

Def Connectives: and, or, not, if-then, if-and-only-if (iff), either-or

Def Quantifiers: for-all (all), exists (some), none

(Different fonts/boldface/colors are often used initially to better distinguish between e.g. "and" in enumerations and "and" as logical connective.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

| Situation | | not | $A$ | $A$ | If $A$, | $A$ | Either $A$ |
|---|---|---|---|---|---|---|---|
| | | $A$ | and | or | then $B$ | iff | or $B$ |
| $A$ | $B$ | | $B$ | $B$ | | $B$ | |
| false | false | true | false | false | true | true | false |
| false | true | true | false | true | true | false | true |
| true | false | false | false | true | false | false | true |
| true | true | false | true | true | true | true | false |

- "Classification" of all situations wrt. $A, B$.

EG For which values of $x \in \mathbb{R}$ (situation; value stored in memory) do you agree to

- "$x = 2$ and $x^2 = 4$."
- "$x = 2$ or $x \neq 2$." ("$x \neq 2$" short for "not $x = 2$")
- "$x = 2$ and $x \neq 2$."
- "If $x = 2$, then $x^2 = 4$."
- "If $x^2 = 4$, then $x = 2$."
- "$x = 2$ if and only if $x^2 = 4$."

For personal use only, no commercial use. TUM CIT Department of Computer Science

| Situation $A$ | Situation $B$ | not $A$ | $A$ and $B$ | $A$ or $B$ | If $A$, then $B$ | $A$ iff $B$ | Either $A$ or $B$ |
|---|---|---|---|---|---|---|---|
| false | false | true | false | false | true | true | false |
| false | true | true | false | true | true | false | true |
| true | false | false | false | true | false | false | true |
| true | true | false | true | true | true | true | false |

- The conditional proposition "if $A$, then $B$" is only false in a situation in which $A$ is true but (and) $B$ is false.
  - $B$ does not matter in a situation in which $A$ is false ("hypothetical").
  - "if-then" is called conditional: the propositions denoted by $A$ and $B$ are also called "antecedent" ("comes first") resp. "consequent" ("follows after").
  - Typical examples are rules/laws/guidelines ((social) contract, program) which can be violated in a concrete situation.

    (See e.g. the Wason selection task; note: "knowing" the rules does not mean to understand (the reason for) the rules; e.g. we need to prove that a program does what it is supposed to do; cf. "sufficient" and "necessary" later).

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Connectives

| Situation | | not | $A$ | $A$ | If $A$, | $A$ | Either $A$ |
|---|---|---|---|---|---|---|---|
| | | $A$ | and | or | then $B$ | iff | or $B$ |
| $A$ | $B$ | | $B$ | $B$ | | $B$ | |
| false | false | true | false | false | true | true | false |
| false | true | true | false | true | true | false | true |
| true | false | false | false | true | false | false | true |
| true | true | false | true | true | true | true | false |

- For now: "**syn**" short for "**syn**onymous (same meaning)" (will be made more precise):
  - "if $A$, then $B$" **syn** "$B$ or not $A$" **syn** "if not $B$, then not $A$"
  - "$A$ iff $B$" **syn** "if $A$ then $B$; and if $B$ then $A$" ("iff" short for "if and only if")
  - "either $A$ or $B$" **syn** "not ($A$ iff $B$)"
  - "neither $A$ nor $B$" **syn** "not ($A$ or $B$)" **syn** "(not $A$) and (not $B$)"

(Use brackets to avoid syntactical ambiguities.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

| Situation | | not | $A$ | $A$ | If $A$, | $A$ | Either $A$ |
|---|---|---|---|---|---|---|---|
| | | $A$ | and | or | then $B$ | iff | or $B$ |
| $A$ | $B$ | | $B$ | $B$ | | $B$ | |
| false | false | true | false | false | true | true | false |
| false | true | true | false | true | true | false | true |
| true | false | false | false | true | false | false | true |
| true | true | false | true | true | true | true | false |

- More formally, a proposition using variables is called a propositional form:
  - Only after substituting $A, B$ by actual propositions (e.g. the trivial propositions "true", "false") we can agree or disagree (many forms of form).
  - We can also replace $A, B$ by other propositional forms or mathematical expressions to form complex propositions (forms).

    Use brackets to avoid syntactical ambiguity ("I saw a man on the hill ...").

EG "If ((if $A$, then $B$) and $A$), then $B$".

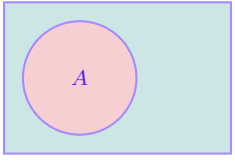
  - Propositional forms can also be illustrated using Venn diagrams.

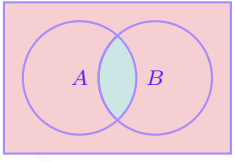
The same proposition can take different "forms".

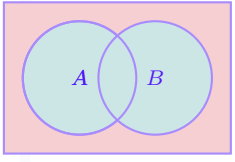
For personal use only — not for commercial use — TCM CUT Department of Computer Science

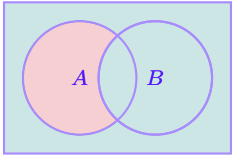
# Connectives: truth tables and Venn diagrams

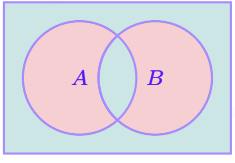
| Situation $A$ | Situation $B$ | not $A$ | $A$ and $B$ | $A$ or $B$ | If $A$, then $B$ | $A$ iff $B$ | Either $A$ or $B$ |
|---|---|---|---|---|---|---|---|
| false | false | true | false | false | true | true | false |
| false | true | true | false | true | true | false | true |
| true | false | false | false | true | false | false | true |
| true | true | false | true | true | true | true | false |

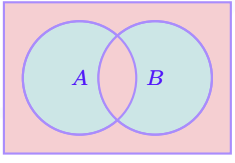


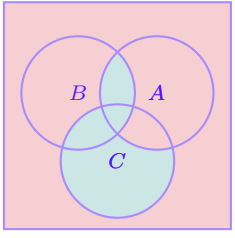




For personal use only, no commercial use – TUM CIT Department of Computer Science

# If-then-else vs. conditional vs. biconditional

- Analogously, truth tables and Venn diagrams can be constructed for three or more propositions (variables), e.g. if-then-else:

| Situation | | | if $A$ then $B$ else $C$ | if $A$ then $B$ else true | if $A$ then $B$ else not $B$ | if $A$ then $B$ and if not $A$ then $C$ |
|---|---|---|---|---|---|---|
| $A$ | $B$ | $C$ | | | | |
| false | false | false | false | true | true | false |
| false | false | true | true | true | true | true |
| false | true | false | false | true | false | false |
| false | true | true | true | true | false | true |
| true | false | false | false | false | false | false |
| true | false | true | false | false | false | false |
| true | true | false | true | true | true | true |
| true | true | true | true | true | true | true |




For personal use only, no commercial use – TUM CIT Department of Computer Science

# If-then-else vs. conditional vs. biconditional

- Analogously, truth tables and Venn diagrams can be constructed for three or more propositions (variables), e.g. if-then-else:

| Situation $A$ | $B$ | $C$ | if $A$ then $B$ else $C$ | if $A$ then $B$ else true | if $A$ then $B$ else not $B$ | if $A$ then $B$ and if not $A$ then $C$ |
|---|---|---|---|---|---|---|
| false | false | false | false | true | true | false |
| false | false | true | true | true | true | true |
| false | true | false | false | true | false | false |
| false | true | true | true | true | false | true |
| true | false | false | false | false | false | false |
| true | false | true | false | false | false | false |
| true | true | false | true | true | true | true |
| true | true | true | true | true | true | true |

- "if-then" is called (material) conditional (subjunction): the truth value of $B$ only matters under the condition (assumption; in the case) that $A$ is true.
- "if-and-only-if" is called (material) biconditional.

⚠ Causality or temporal dependencies do not matter for propositional logic:
"If pigs can fly, then $\pi \in \mathbb{R} \setminus \mathbb{Q}$" is true (w.r.t. the standard definition of $\pi, \mathbb{R}, \mathbb{Q}$).

⚠ Programming languages usually only know "if $A$ is true, then do $b$, else do $c$" for a proposition (condition, guard) $A$ and instructions $b, c$.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Conditional, rules, programs and contracts

EG In contrast to propositional logic, a program can assign a new meaning/value to a variable. (But we can "unfold" programs and then use propositional logic.)

A conditional statement of the form "if $A$, then $b$ else $c$" then combines a proposition $A$ with two instructions $b, c$ (a guarded command/instruction, "guideline/rule").

```
if x is even: # proposition ``x is even'': either true or false
  x := x/2 # instruction: change meaning/value of x to the value of x/2
else:
  x := 3x + 1 # instruction: change meaning/value of x to the value of 3x + 1
```

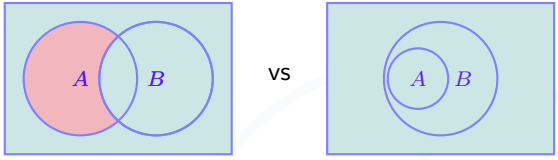
EG A contract is usually a list of clauses. A clause is a conditional statement (a rule) like "if student tries to cheat in exam, student will be exmatriculated".

- ▷ To satisfy a contract means to satisfy every clause ("clause 1 and clause 2 and ... and clause $n$ have to be satisfied").
- ▷ To breach a contract means that at least one clause is not satisfied ("clause 1 or clause 2 or ... or clause $n$ has to be false.")
- ▷ If the antecedent of every clause is false, the contract is trivially satisfied.

EG In logic programming, programs and contracts coincide (cf. "resolution").

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Sufficient vs. necessary, implies




- In general "if $A$, then $B$" is not valid.

  EG "If $x \in \mathbb{Z}$, then $x/2 \in \mathbb{Z}$" is true for $x := 2$, but false for $x := 1$.

  I.e. being in a situation in which "$x \in \mathbb{Z}$" is true does not suffice to conclude that also "$x/2 \in \mathbb{Z}$" is true (in other words: "$x/2 \in \mathbb{Z}$" is not necessary for "$x \in \mathbb{Z}$".)

  But for specific $A, B$, "if $A$, then $B$" can be valid.

  EG "If $x/2 \in \mathbb{Z}$, then $x \in \mathbb{Z}$" is valid.

  I.e. being in a situation in which "$x/2 \in \mathbb{Z}$" is true suffices to conclude that also "$x \in \mathbb{Z}$" is true (in other words: "$x \in \mathbb{Z}$" is necessary for "$x/2 \in \mathbb{Z}$".)

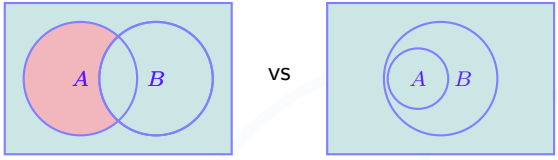
  EG Both "if $x \in 2\mathbb{Z}$, then $x/2 \in \mathbb{Z}$" and "if $x/2 \in \mathbb{Z}$, then $x \in 2\mathbb{Z}$" are valid.

  I.e. both $x/2 \in \mathbb{Z}$ and $x \in 2\mathbb{Z}$ are sufficient and necessary for the other.

(Note that "$\mathbb{Z}$" implicitly refers to the definition of the integers.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Sufficient vs. necessary, implies




- Given that "if $A$, then $B$" is valid (for specific $A, B$), one also says: "$A$ is sufficient for $B$" resp. "$B$ is necessary for $A$".
  - Frequently also: "$A$ implies $B$", "$B$ whenever $A$", "$B$ follows (logically) from $A$"

  Should "$A$ iff $B$" be valid, then "$A$ is sufficient and necessary for $B$".

  ($A$ iff $B$ **syn** if $A$, then $B$; and if $B$, then $A$.)
- Mathematical theorems and lemmata are valid propositions of the form "if $A$, then $B$" or "$A$ iff $B$". (The validity must be proven/argued, see later.)

  To indicate a valid "iff" sometimes "**iff**" is used.

EG (i) ($A$ iff $B$) **iff** ($B$ iff $A$); (ii) (if $A$, then $B$) **iff** ($B$ or not $A$).

- "**iff**" is a special case of "**syn**" for propositional forms.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Restricted quantifiers

- "If all bandersnatches are borogoves, and all borogoves are slithy, then all bandersnatches are slithy."
- How to read "all/some of kind ("type", "property") $1$ are also of kind $2$"?
- Formally, restricted quantifiers:
  - "All $x$ with $R(x)$ also satisfy $P(x)$." (have the property $P$, "are" $P$)
    "For all $x$, for which $R(x)$ is true, also $P(x)$ is true."
    "For every object (thing) $x$, if $R(x)$ is true, then also $P(x)$ is true."
  - "Some $x$ with $R(x)$ also satisfy $P(x)$."
    "There exists some $x$, for which $R(x)$ is true, and further $P(x)$ is true."
    "There is some object (thing) $x$ that satisfies both $R(x)$ and $P(x)$."

  where $R, P$ denote predicates (parameterized propositions specifying a property), e.g.:

  - "$x$ is a bandersnatch", "$x$ is a borogove", "$x$ is slithy"
  - "$x = 2$", "$x^2 = 2$", "$x^2 = -1$", "$x > y$", "$x \neq y$", "$x \in \mathbb{Q}$", "$x \in \mathbb{R}$"

  (Formally e.g. "is a bandersnatch" and "$>$ (greater)" are the actual predicates.)

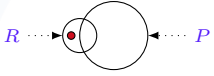
For personal use only, no commercial use – TUM CIT Department of Computer Science

# Restricted quantifiers and Venn (resp. Euler) diagrams

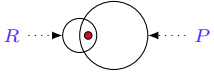
- "All $x$ with $R(x)$ are $P(x)$" **iff** "for all $x$, if $R(x)$, then $P(x)$"




- "not all $x$ with $R(x)$ are $P(x)$" **iff** "there exists $x$ with $R(x)$ and not $P(x)$"




- "Some $R(x)$ are $P(x)$" **iff** "There exists $x$ with $R(x)$ and $P(x)$"




- "no $x$ with $R(x)$ is $P(x)$" **iff** "for all $x$, if $R(x)$, then not $P(x)$"




- Compare: $R(x) :=$ "$x \in \mathbb{Q}$" vs. $R(x) :=$ "$x \in \mathbb{R}$" for $P(x) :=$ "$x^2 \neq 2$"

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Restricted quantifiers

- "Some $x$ with $R(x)$ are $P(x)$" is only true **iff** we can find a concrete object $d$ for which $R(d)$ and $P(d)$ is true ($d$ is then an example or witness).
  - More formally: "There exists $x$ with $R(x)$ and $P(x)$"
- "All $x$ with $R(x)$ are $P(x)$" is only false **iff** we can find a concrete object $d$ for which $R(d)$ and not $P(d)$ is true ($d$ is then a counterexample).
  - More formally: "for all $x$, if $R(x)$, then $P(x)$"

EG "There exists a rational number $x$ with $x^2 = 2$." (see Euclid later)
vs "There exists a real number $x$ with $x^2 = 2$."

EG "For all rational numbers $x$ we have $x^2 \neq 2$."
vs "For all real numbers $x$ we have $x^2 \neq 2$."

EG "For all real numbers $x$, if $x = \sqrt{2}$, then $x^2 = 2$."
vs "For all real numbers $x$, if $x^2 = 2$, then $x = \sqrt{2}$."
vs "For all real numbers $x$, if $x^2 = 2$, then $x = -\sqrt{2}$ or $x = \sqrt{2}$."
vs "For all real numbers $x$: $x^2 = 2$ iff $x = -\sqrt{2}$ or $x = \sqrt{2}$."

⚠ In natural language the "for-all" often moves to the end, e.g.:
"$x^2 \neq 2$ for all rational numbers $x$."

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Restricted quantifiers

- "Some $x$ with $R(x)$ are $P(x)$" is only true **iff** we can find a concrete object $d$ for which $R(d)$ and $P(d)$ is true ($d$ is then an example or witness).
  - More formally: "There exists $x$ with $R(x)$ and $P(x)$"
- "All $x$ with $R(x)$ are $P(x)$" is only false **iff** we can find a concrete object $d$ for which $R(d)$ and not $P(d)$ is true ($d$ is then a counterexample).
  - More formally: "for all $x$, if $R(x)$, then $P(x)$"
- Related by means of negation: existence vs. absence of a (counter-)example
  - "not all $x$ with $R(x)$ are $P(x)$" **iff** "some $x$ with $R(x)$ are not $P(x)$"
  - "no (not some) $x$ with $R(x)$ is $P(x)$" **iff** "all $x$ with $R(x)$ are not $P(x)$"

EG "Not for all real numbers $x$ is it true that $x^2 = -1$." **iff**
"There exists a real number $x$ with $x^2 \neq -1$."

EG "There is no real number $x$ with $x^2 = -1$" **iff**
"No real number $x$ satisfies $x^2 = -1$" **iff**
"It is not true that there is a real number $x$ with $x^2 = -1$" **iff**
"For every real number $x$ we have $x^2 \neq -1$."

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Validity and proofs

- A proof is a sequence of arguments with the goal to argue (show, prove) that a certain proposition is valid (always true).

EG "If $A$, then $A$" is valid. (This is a proposition with a propositional form as subject.)

- Proof (argument, argumentation, reasoning):

  Whatever proposition $A$ might stand for, it is either true or false.

  Consider a situation in which $A$ is true: by definition of the semantics of "if-then", we only need to check that $A$ is true which it is by assumption.

  Consider a situation in which $A$ is false: by definition of the semantics of "if-then", the truth value of "if $A$, then $A$" is true.

- In simple cases a truth table (an "exhaustive" case distinction) suffices:

EG Show that "if ((if $A$, then $B$) and $A$), then $B$" is valid.

- In general we have to break the reasoning down into smaller (valid) arguments ("steps") using the definitions of the connectives and quantifiers.

- Typically, a "proof step" is of the form "if you have agreed to everything said so far, then in addition you necessarily have to agree to the following (as [explication])" which is called a (deductive) inference.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Proofs

- Claim: "If all bandersnatches are borogoves, and all borogoves are slithy, then all bandersnatches are slithy." is valid.
- For suitable predicates $P$, $Q$, $R$ we want to prove

| | | |
|---|---|---|
| If | (A) | for all $x$: if $P(x)$, then $Q(x)$, |
| and | (B) | for all $y$: if $Q(y)$, then $R(y)$, |
| then | (C) | for all $z$: if $P(z)$, then $R(z)$. |

- Proof: (Draw the diagrams.)
  - It suffices to consider a situation in which (A) and (B) are true (else done).
  - We need to show that then (C) is true, i.e. that there is no object (thing) $d$ s.t. $P(d)$ is true and $R(d)$ is false (definition of "for-all" and "if-then").
  - Let $d$ denote any such object $d$ of which we only know that $P(d)$ is true. Then:
    - As $P(d)$ and (A) are true, also $Q(d)$ must be true (from (A): "if $P(d)$, then $Q(d)$").
    - As $Q(d)$ and (B) are true, also $R(d)$ must be true (from (B): "if $Q(d)$, then $R(d)$").
  - Hence, whenever (A) and (B) are both true, "every thing" which has property $P$ has also property $R$, i.e. also (C) is true independent of the (subjective) interpretation of $P, Q, R$ ("bandersnatch", "borogoves", "slithy").

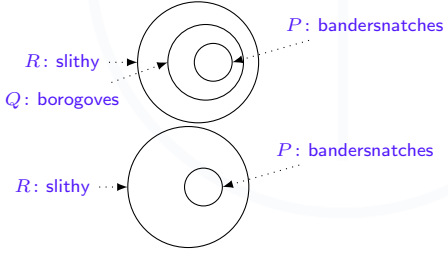
For personal use only, no commercial use – TUM CIT Department of Computer Science

| | | | |
|---|---|---|---|
| If | | Ⓐ | for all $x$: if $P(x)$, then $Q(x)$, |
| | and | Ⓑ | for all $y$: if $Q(y)$, then $R(y)$, |
| then | | Ⓒ | for all $z$: if $P(z)$, then $R(z)$. |

▷ If Ⓐ resp. Ⓑ is true:




▷ Whenever Ⓐ and Ⓑ are true, in particular Ⓒ is true:




For personal use only, no commercial use – TUM CIT Department of Computer Science

EG "If all pigs can fly and Miss Piggy is a pig, then Miss Piggy can fly" has the structure

| | | | |
|---|---|---|---|
| If | | (A) | for all $x$: if $P(x)$, then $Q(x)$, |
| | and | (B) | $P(\mathrm{mp})$, |
| then | | (C) | $Q(\mathrm{mp})$. |

for $P(x) :=$ "$x$ is a pig", $Q(x) :=$ "$x$ can fly", $\mathrm{mp} :=$ "(some object known as) Miss Piggy". (As before read ":=" as "let LHS stand for/denote RHS")

Use Venn diagrams to visualize how $P, Q, \mathrm{mp}$ might be related to each other; then decide if the statement is valid.

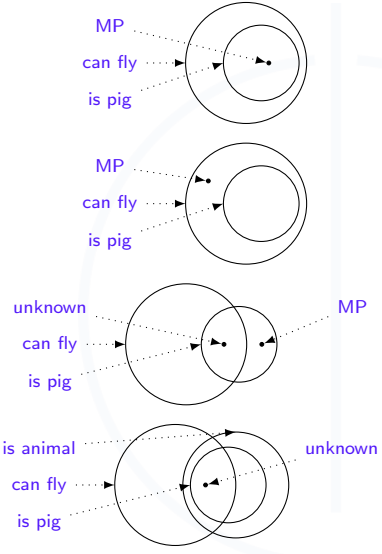
EG Similarly decide which of the following is valid:

- "If all pigs can fly and Miss Piggy can fly, then Miss Piggy is a pig".
- "If some pigs can fly and Miss Piggy is a pig, then Miss Piggy can fly".
- "If some pigs can fly and all pigs are animals, then some animal can fly".

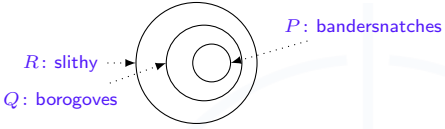
- These particular form of arguments are called syllogism (Greek: "conclusion, inference") and attributed to Aristotle.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Proofs and inferences as Euler diagrams




For personal use only, no commercial use – TUM CIT Department of Computer Science




▷ A (deductive) inference is an argument (1.1) of the form

$$\frac{\begin{array}{lll}\text{(If)} & \textsf{P1} & \ldots,\\ \text{(and)} & \textsf{P2} & \ldots,\end{array}}{\begin{array}{lll}\text{(then)} & \textsf{C} & \ldots.\end{array}}$$

where

- ▷ P1 , P2 are called premis(s)es.

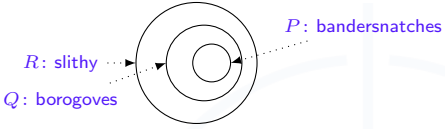
  In general, "P1 and P2 and ... and Pn " for some $n \in \mathbb{N}$.

  (In principle, two premisses $A, B$ can be combined into a single premiss "$A$ and $B$", but typically "$A$ and $B$" is treated as two premisses $A, B$.)

- ▷ C is called the actual conclusion or consequence.

("con-cludere": essentially "both (premises) locked together", "con-sequi": essentially "from both (premises) together follows"; "prae-mittere": essentially "to send ahead".)

For personal use only, no commercial use – TUM CIT Department of Computer Science




- Typically abbreviated to ("P1 ; P2 ; thus C")




"if P1 and P2 , then C" is usually not written explicitly in an argument/proof.

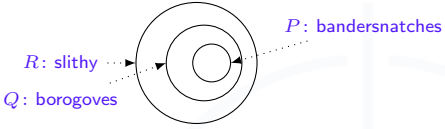
As in the "Bandersnatch example" it must be read as "Assume $A$ is true; further assume $B$ is true; then (we claim) $C$ is necessarily true".

("ad-sumere": "to take up, to accept, to adopt, (to buy)" (an idea))

- The "line" ("rule") corresponds to a "thus", "hence", "it follows", "subsequently", "therefore", "followingly", "consequently", "accordingly".

(Conceptually marks the point up to which the argument/deduction has been accepted.)

For personal use only, no commercial use – TUM CIT Department of Computer Science



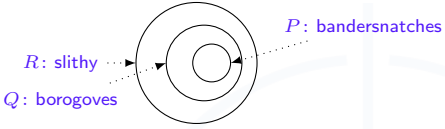

- Aristotle's Analytica Priora is considered to be the first work studying when an argument is valid.

  In the first part he studies arguments consisting only of two premisses (as part of a longer argument, a "step") with both premisses and conclusion of the form "all/some/none $x$ with $P(x)$ are (not) $Q(x)$"' and the premisses "connected" by means of a shared (unary) predicate (see "'Bandersnatch"').

  This particular simple form of arguments is often called a syllogism but "syllogism" can have also the more general meaning of "logical inference/deduction, valid argument" (see here).

  ("Syllogismus": "$\sigma\upsilon\nu$-" and "$\lambda o\gamma o\varsigma$", "together-counted/computed/thought".)

For personal use only, no commercial use – TUM CIT Department of Computer Science




- ▷ In general, premisses are assumptions (might be unproven, invalid, unsatisfiable), proven valid statements (e.g. theorems, lemmata), axioms (principles, basic assumptions, not necessarily provable, part of a definition), rules (either as lemma or axiom), …

  (Depending on the author/source/area of research, there can be subtle differences in the precise use of all these words, see also postulates, hypotheses, conjectures, laws.)

- ⚠ If the inference is valid, the premisses together ("P1 and P2") are sufficient for the conclusion, while the conclusion is necessary for truth of all the premisses.

  If the inference (argument) is invalid, then C can still be false in a situation in which both P1 and P2 are true.

- ⚠ The conclusion can become an additional premiss for the subsequent inferences.

- ▷ Valid inferences give rise to "rules of inference" which allow us to "calculate/compute" with proofs (see proof calculus).

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Some rules of inference

$$\frac{\text{if } A \text{ then } B \quad A}{B} \qquad \frac{\text{if } A \text{ then } B \quad \text{not } B}{\text{not } A} \qquad \frac{A \text{ or } B \quad \text{not } B}{A} \qquad \frac{\text{if } A \text{ then } B \quad \text{if } B \text{ then } C}{\text{if } A \text{ then } C}$$

$$\frac{\text{if } A \text{ then } C \quad \text{if } B \text{ then } C \quad A \text{ or } B}{C} \qquad \frac{\text{if not } B \text{ then not } A}{\text{if } A \text{ then } B} \qquad \frac{\text{if not } A \text{ then } B \quad \text{if not } A \text{ then not } B}{A}$$

$$\frac{\text{All } x \text{ with } P(x) \text{ are } Q(x) \quad P(d)}{Q(d)} \qquad \frac{\text{All } x \text{ with } P(x) \text{ are } Q(x) \quad \text{Some } x \text{ are } P(x)}{\text{Some } x \text{ are } Q(x)} \qquad \frac{\text{All } x \text{ with } P(x) \text{ are } Q(x) \quad \text{All } x \text{ with } Q(x) \text{ are } R(x)}{\text{All } x \text{ with } P(x) \text{ are } R(x)}$$

- "If you agree to all of above the line, you must agree also to that below."
- Note that we can always add more premises as these can only reduce the potential situations.
- The order of the premises does not matter.
- The formally correct treatment of rules of inference involving quantifiers requires to introduce the concepts of free and bound variables (which we won't discuss now).
- See the previous examples and the tutorials. (Check the validity using a truth table.)
- Again, the premises and conclusion on their own can be false or even unsatisfiable.

For personal use only, no commercial use – TUM CIT Department of Computer Science

EG Consider the equation system:

$$3x + 4y = 0 \qquad -2x + 3y = 17$$

Assume that there exist concrete numbers $x, y \in \mathbb{R}$ which satisfy both (and) equations at the same time. Then, (by definition of $\mathbb{R}$), it follows that necessarily

(A) $3x + 4y = 0$
(B) $-2x + 3y = 17$
---
(solve (A) for $y$)
(C) $y = -0.75x$
---
(substitute RHS of (C) for $y$ in (B) and simplify)
(D) $-4.25x = 17$
---
(solve (D) for $x$)
(E) $x = -4$
---
(substitute RHS of (E) for $y$ in (C) and simplify)
(F) $y = 3$

(Check yourself if/that each step is a valid inference in $\mathbb{R}$.)

Hence, necessarily $x = -4$ and $y = 3$ (unique solution).

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Proof by contraposition/contrapositive and contradiction

- ▷ Proof by contraposition (contrapositive): contraposition refers to
  - ▷ "If $A$ then $B$" **iff** "If not $B$, then not $A$" (sufficient/necessary)
  - ▷ "All $x$ with $P(x)$ are $Q(x)$" **iff** "All $x$ with not $Q(x)$ are not $P(x)$"

  Sometimes it is easier (more natural) to prove the validity of the contraposition.

EG Instead of "all $x$ with $x^2 = -1$ are not real numbers"
prove "all real numbers $x$ satisfy $x^2 \neq -1$" (next).

- ▷ Proof by contradiction: To prove the validity of some proposition $A$, assume a situation in which $A$ is false, i.e. not $A$ is true, and then show that some (arbitrarily chosen) proposition $B$ and its negation would need to be true in this (hypothetical) situation, too. ("if (if not $A$, then $B$ and not $B$), then $A$" is valid.)

EG To show "all real numbers $x$ satisfy $x^2 \neq -1$"
assume a situation in which "some real number $x$ satisfies $x^2 = -1$" is true,
i.e. assume that $x_0$ is a concrete real number with $x_0^2 = -1$.
Then $-1 = x_0^2 = |x_0^2| = |-1| = 1 \neq -1$ in $\mathbb{R}$ (contradiction).

For personal use only, no commercial use – TUM CIT Department of Computer Science

# $\sqrt{2}$ is not rational

- (Definition) $\sqrt{2}$ denotes the (only) positive real number $x$ with $x^2 = 2$.
- Claim: Every rational number $x \in \mathbb{Q}$ satisfies $x^2 \neq 2$.
- Proof by contradiction:
  - Assume there is some rational $x_0$ with $x_0^2 = 2$.
  - Then there exist integers $m, n$ with $x_0 = \frac{m}{n}$. (WLOG: $x_0 > 0$.)
  - We may assume that $m, n$ are completely canceled, i.e. $\gcd(m, n) = 1$
  - Hence $m^2 = 2n^2$.
  - Hence $2$ is a divisor of $m^2$.
  - Hence $2$ is a divisor of $m$ (requires Euclid's lemma).
  - Hence $m = 2k$ and also $2k^2 = n^2$.
  - Hence $2$ is a divisor of $n^2$ and also $n$.
  - Thus, $1$ is not the greatest common divisor of $m, n$.

(See also [here] how Terrence Tao is using the proof assistant Lean ("old-school AI") together with an LLM ("modern AI") to write a formal proof.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Theorem, lemma, corollary, definition, ...

- There are several words which are used in mathematics to refer to a (proven valid) proposition (statement) but differ in the "preceived importance":

  **Thm** Theorem: important, often used and/or rather surprising result.

  **Lem** Lemma: less important or "auxiliary" result for proving a theorem.

  (Often, the proof of a theorem is split into several simpler/shorter to prove lemmata but some lemmata later turned out to be more important than the actual theorem ...).

  **Cor** Corollary: easy to prove variant/consequence of a theorem/lemma.

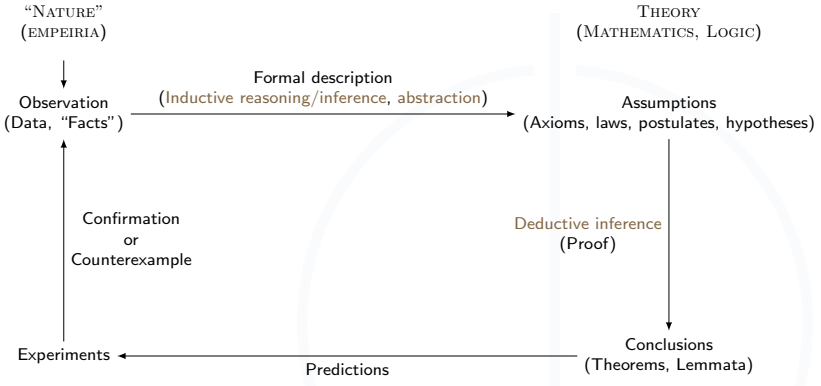
  **Fct** Fact: rather obvious result (proof by intimidation ...).

  **Def** Definition: precise explanation (explication) of the meaning of a word (term) in the current context. (Might require to prove well-definedness.)

  Axioms can be a part of definitions: axioms describe (postulate) properties characterizing a certain class of mathematical structures (see e.g. $\mathbb{N}$ and groups later).

(Some authors also use "proposition" in the sense of "theorem of lesser importance". The precise use depends on the author; e.g. in the thereom prover Isabelle Theorem and Lemma are interchangable. See e.g. also wikipedia.)

For personal use only, no commercial use – TUM CIT Department of Computer Science




⚠ simplified

- Cf. e.g. the history of gravity: Aristotle, Galilei, Newton, Einstein, ...
- Physics: $F = m\frac{d^2s}{dt^2}$, $F = mg$ (Newton's "laws"; free fall in vacuum)

  Math&logic: $s(t) = s_0 + v_0 t + \frac{g}{2}t^2$ (Newton/Leibniz et al.)

  ("AI": machine learning tries to automatize inductive reasoning (e.g. Gauss/least-squares) while proof assistants try to semi-automatiz deductive reasoning.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

*What do we mean by "understanding" something? We can imagine that this complicated array of moving things which constitutes "the world" is something like a great chess game being played by the gods, and we are observers of the game. We do not know what the rules of the game are; all we are allowed to do is to watch the playing. Of course, if we watch long enough, we may eventually catch on to a few of the rules. The rules of the game are what we mean by fundamental physics.*

*Even if we knew every rule, however, we might not be able to understand why a particular move is made in the game, merely because it is too complicated and our minds are limited. If you play chess you must know that it is easy to learn all the rules, and yet it is often very hard to select the best move or to understand why a player moves as he does. So it is in nature, only much more so; but we may be able at least to find all the rules.*

*Actually, we do not have all the rules now. (Every once in a while something like castling is going on that we still do not understand.) Aside from not knowing all of the rules, what we really can explain in terms of those rules is very limited, because almost all situations are so enormously complicated that we cannot follow the plays of the game using the rules, much less tell what is going to happen next. We must, therefore, limit ourselves to the more basic question of the rules of the game. If we know the rules, we consider that we "understand" the world. (Richard Feynmann: "The Feynman Lectures on Physics, Volume I [src]")*

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Remark: notation for connectives and quantifiers



| | formal logic (DS) | | Python (C++) | C, C++, Java |
|---|---|---|---|---|
| true | (trivial tautology) | true, $\top$ | `True` | `true` |
| false | (trivial contradiction) | false, $\bot$ | `False` | `false` |
| not | negation | $\neg A$ | `not A` | `!A` |
| or | disjunction | $(A \vee B)$ | `A or B` | `A\|\|B` |
| and | conjunction | $(A \wedge B)$ | `A and B` | `A&&B` |
| if-then | conditional, subjunction | $(A \to B)$ | `not A or B` | `!A\|\|B` |
| if-and-only-if | biconditional | $(A \leftrightarrow B)$ | `A==B` | `A==B` |
| either-or | exclusive or, (e)XOR | $(A \oplus B)$ | `A!=B` | `A!=B` |
| all | universal quantifier | $\forall x F$ | `all` | |
| some | existential quantifier | $\exists x F$ | `any` | |

- ⚠ The symbols $\wedge, \vee, \neg, \to, \exists, \forall$ do not serve as abbreviations in logic, but for better distinction between the object and the actual argument:

  "The prop. form 'if $A$, then $A$' is valid as ..." vs. "$(A \to A)$ is valid as ..."

- ▷ The "correct" use of the symbols requires to define both syntax and semantics. (Later in DS.)

- ⚠ Specifically the arrow symbols are "overloaded": some authors use both $\to$ and $\Rightarrow$ for the conditional, some use $\Rightarrow$ to indicate validity, some use $\to$ for a substitution rule and $\Rightarrow$ for the induced Semi-Thue system, and so on and so forth – their meaning is highly context-dependent.

- ▷ Mathematical texts (outside of logic) use symbolic logic often informally and only in addition to clarify the structure of a proposition to avoid syntactical ambiguities. Only use symbolic logic if explicitly asked to do so. Your lecturer will not guess what you try to say.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Connectives in programming languages



```
%# use e.g. colab
for A in [False, True]:
  for B in [False, True]:
    print("\t".join(str(x) for x in (
      A, B, "|", not A, A and B, A or B, not A or B, (not A or B) and (not B or A), (B if A else True)
    )))
```

- Python actually only knows the connectives `and`, `or`, `not`.
- If `A`, `B` are truth values, then `A==B` and `A!=B` can be used to express if-and-only-if and either-or. (Their actual meaning is "is equivalent" resp. "is not equivalent".)
- Most programming languages know "`if-then(-else)`" only for conditional statements and expressions (branching):
  - In (imperative) programming language read as

    "(only) `if` condition is true, `then` **do** something, `else` **do** alternative"
  - Python also understands "`B` **`if`** `A` **`else`** `True`" which is short for "if $A$ is true, then return $B$, else return True" and the closest thing to the logical conditional.
- See the lectures on Python later.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Predicates and restricted quantifiers in Python

```
S = {0, 1, 2, 3, 4, 5, 6, 7, 8}                  # Let S be the set {0,1,...,8}
print(1, all(x > 0 for x in S))                  # For all x ∈ S we have x > 0
print(2, any(x > 0 for x in S))                  # There is some x ∈ S with x > 0
print(3, not all(not x > 0 for x in S))          # Not for all x ∈ S is x > 0 false
print(4, not any(not x > 0 for x in S))          # There is no x ∈ S for which x > 0 is false
print(5, any(x > 0 and x < 3 for x in S))        # There is some x ∈ S with 0 < x < 3
print(6, all(any(x == y for y in S) for x in S)) # for all x ∈ S there is some y ∈ S with x = y
print(7, any(all(x == y for x in S) for y in S)) # There is some y ∈ S s.t. for all y ∈ S we have x = y
```

- Python also knows all and some as `all` and `any` ("is there any ...?").
  - The restriction only moves to the end: "All $x \in S$ are $P(x)$" becomes `all(P(x) for x in S)`.
  - Predicates e.g.: `x>0`, `x in S`, `x<3`, `x>0 and x<3`, `not x>0`, `x % y != 0`

⚠ `any(x >= y for y in S)` on its own will cause Python to compain "name 'x' is not defined".

- `x` is a free variable as it is free of any meaning.
- Nested within `all(any(x >= y for y in S) for x in S))` it acts as a predicate with `x` now bound to the meaning that it is an element of `S`.
- Formal logic requires to introduce all these concepts and acts as foundation of programming languages.

For personal use only, no commercial use – TUM CIT Department of Computer Science

1. Basic concepts
   - Basic concepts of logic
   - Sets
     - Basic terminology and concepts
     - Russell's paradox
     - Subsets, equality, set operations
     - Sets of sets (nested sets)
     - Sets in Python
   - Sequences, tuples, and words
   - The natural numbers, proof by induction
   - Relations
   - Functions

For personal use only, no commercial use – TUM CIT Department of Computer Science



Georg Cantor

[source]

§ 1.

**Der Mächtigkeitsbegriff oder die Cardinalzahl.**

Unter einer ‚Menge' verstehen wir jede Zusammenfassung $M$ von bestimmten wohlunterschiedenen Objecten $m$ unsrer Anschauung oder unseres Denkens (welche die ‚Elemente' von $M$ genannt werden) zu einem Ganzen.

[source]

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Basic concept of a set

- The concept "'set"' refers to the idea of forming a collection of unambiguously/uniquely distinguishable objects e.g. based on a property common to each of these distinct objects.
  - Every car has a unique vehicle identification number (VIN).
  - Often, the collection of all cars of the same manufacturer is considered.
- When defining a set the order of objects or mentioning an object several times does not matter.
  - That is, in a set there is no first, second, third, etc. object; also "'contained twice"', "'contained thrice"' etc. is not defined for a set (only "'not contained"' and "'contained"').
- Further, the basic opterations for forming new sets from given sets are important: Union, intersection, difference, ...

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Basic definitions

**Def** Let $x$ be an object and $M$ a set. Read:

- ▷ "$x \in M$" as "the object $x$ is an element of $M$".
  (Also: "$x$ is a member of $M$", "$x$ is in $M$" (Python), "$x$ is contained in $M$".)
- ▷ "$x \notin M$" as "the object $x$ is not an element of $M$".

("$M$" for "Menge".)

**Def** For every set $M$ and every object $x$ either $x \in M$ or $x \notin M$ is true.

**Def** The empty set is denoted by $\{\}$ or $\emptyset$.
By definition $x \notin \emptyset$ is true for every object $x$.
(I.e. there is no object $x$ with $x \in \emptyset$.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Infinite sets of numbers known from school

Fct We simply take as a fact that the following sets known from school are well-defined:

- $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$ set of the natural numbers including $0$.
  $\mathbb{N} = \{1, 2, 3, \ldots\}$ set of the positive integers.
- $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$ set of the integers.
- $\mathbb{Q} = \{p/q \mid p \in \mathbb{Z}, q \in \mathbb{N}\}$ set of the rational numbers.
- $\mathbb{R} = \{\pm(n + \sum_{i=1}^{\infty} d_i 10^{-i}) \mid n \in \mathbb{N}_0, i \in \mathbb{N}, d_i \in \{0, 1, ..., 9\}\}$ set of the real numbers.

⚠ Formally, $\mathbb{N}$ and $\mathbb{N}_0$ have to be defined e.g. using the axiom of infinity or the Peano axioms (see the section on the natural numbers).

$\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$ then can be constructed from $\mathbb{N}_0$ (via quotients and suprema).

- There are several ways to define additional sets:
  - By means of an explicit (finite) list.
  - By means of "separation and replacement" based on an already defined set.
  - By means of set-theoretic operations (union, intersection, ...; later).

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Sets: Definition by means of a listing (enumeration)

**Def** $x \in \{x_1, \ldots, x_k\}$ **:iff** $x$ occurs (at least once) in $x_1, \ldots, x_k$.
$x \notin \{x_1, \ldots, x_k\}$ **:iff** $x$ does not occur in $x_1, \ldots, x_k$.

- ▷ Curly brackets "$\{$, $\}$" are used to indicate the start and the end of a set definition by means of an explicit list $x_1, \ldots, x_k$ with commata "," as separator/delimiter.

⚠ In the strict sense, the list must be complete (and thus finite), but often it is also accepted to only indicate the list and give some further explanation:

**EG**
- ▷ $\{\mathbb{N}, \mathbb{N}_0\}$ and $\{\mathbb{N}, \mathbb{N}_0, \mathbb{Z}\}$ also sets.
- ▷ {A,B,...,Z,a,b,...,z} the set of the letters of the Latin alphabet.
- ▷ $\{!, ", \#, \ldots, \sim\}$ the set of all printable ASCII characters (excluding whitespace).

**Def** $[n] := \{1, 2, \ldots, n\}$ for every $n \in \mathbb{N}_0$ with $[0] := \emptyset$.
$\mathbb{Z}_n := \{0, 1, \ldots, n-1\}$ for every $n \in \mathbb{N}_0$ with $\mathbb{Z}_0 := \emptyset$.

⚠ "**:iff**" and "$:=$" to emphasize a definition:

"let the text on the LHS denote/stand for the text on the RHS".

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Sets: Definition by means of separation (selection) and replacement



**Def** $x \in \{y \in A \mid P(y)\}$ **:iff** both $x \in A$ and $P(x)$ are true.

($x \notin \{y \in A \mid P(y)\}$ **iff** $x \notin A$ or ($x \in A$ but $P(x)$ false).)

**Def** $x \in \{f(y) \mid y \in A\}$ **:iff** $x = f(y)$ is true for some $y \in A$.

($x \notin \{f(y) \mid y \in A\}$ **iff** $x \neq f(y)$ for all $y \in A$.)

⚠ It is important that we "select" from an already well-defined set $A$ and that the property (predicate) $P$ resp. construction (function) $f$ is also well-defined (no ambiguities; cf. Russell next).

- ▷ Instead of "|" also ":" is used (e.g. $\{x \in \mathbb{R}\colon |x| < 1\}$).
- ▷ "$\{x \in A \mid P(x), Q(x)\}$" has to be read as "$\{x \in A \mid P(x) \text{ and } Q(x)\}$".
- ▷ Instead of "$\{x \in A \mid P(x)\}$" also "$\{x \mid x \in A, P(x)\}$".

**EG**

- ▷ $A := \{0, 1, 2, 3, 4\}$
- ▷ $B := \{x \in A \mid x \leq 2\}$ or $B := \{x \mid x \in A, x \leq 2\}$
- ▷ $C := \{y \in A\colon y \text{ even}\}$ or $C := \{y\colon y \in A, y \text{ even}\}$
- ▷ $D := \{2x \mid x \in A\}$, $E := \{\{x\} \mid x \in A, x \leq 2\}$

For personal use only, no commercial use – TUM CIT Department of Computer Science

- Sets are also objects and, thus, can be elements of sets themselves, e.g.:

$$\{\, 1, \{1, 2\}, \{1, \{\}\}, \{\{1\}, \{\{2\}\}, 3\} \,\}$$

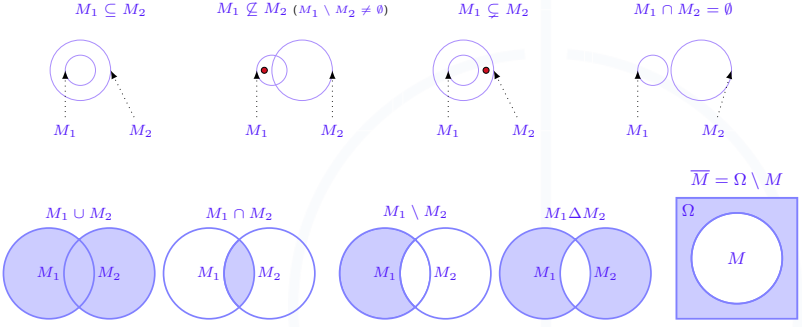
(Think of nested boxes.)

- Question: Is $R \in R$ true for $R := \{x \mid x \notin x\}$?
- Problem: cyclic dependency (also), self-reference of the definition.
- Strengthen the definition of set: Every set $M$ has to satisfy $M \notin M$.
  - More precisely: use either the axiom of regularity (von Neumann) or the axiom of specification (precise definition of how to select from a given set).
  - Then there is no "set of all sets", in particular $\{x \mid x \notin x\}$ does not define a set.
  - Introduce the term "class of all sets" for the "collection of all sets".

(For more details see Zermelo-Fraenkel set theory, type theory.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

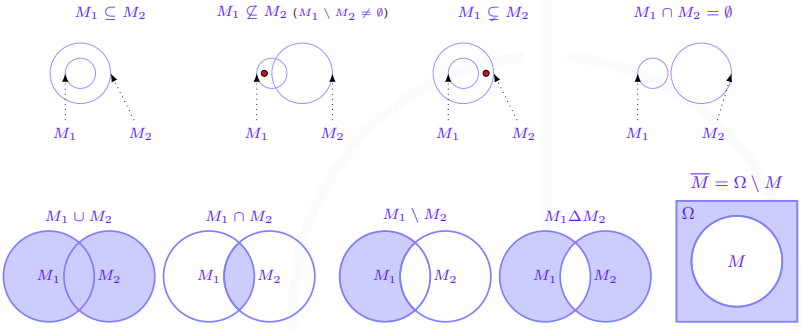
# Operations on sets and relations between sets




- Venn/Euler diagrams as before: sets represented as areas, typically as interior of closed curves (circles, rectangles, ellipses, ...).
- The definitions of the basic set operations and relations are based on the definitions of the connectives and quantifers.
- Bounding rectangle represents "universal set (universe)" $\Omega$.
  Only required for complement.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Operations on sets and relations between sets

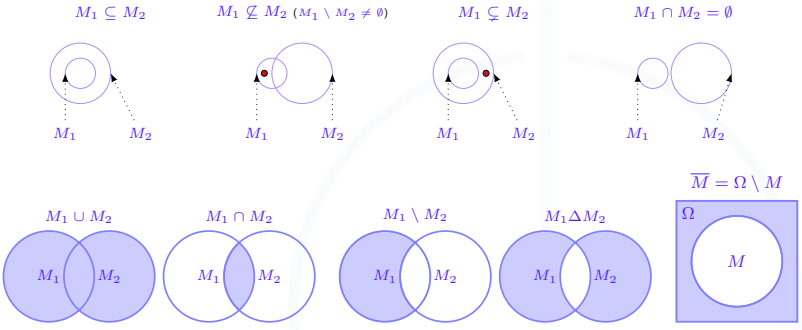



**Def** Let $M_1, M_2$ denote arbitrary (well-defined) sets (Mengen). Then:

- $M_1 \subseteq M_2$ **:iff** for every $x \in M_1$ also $x \in M_2$ ($M_1$ is a subset of $M_2$).
  $M_1 \not\subseteq M_2$ **:iff** for some $x \in M_1$ we have $x \notin M_2$ ($M_1$ is not a subset of $M_2$).
- $M_1 \subsetneq M_2$ **:iff** $M_1 \subseteq M_2$ and $M_2 \not\subseteq M_1$ ($M_1$ is a strict subset of $M_2$).
- $M_1 = M_2$ **:iff** $M_1 \subseteq M_2$ and $M_2 \subseteq M_1$ ($M_1, M_2$ are equal).
  $M_1 \neq M_2$ **:iff** $M_1 \not\subseteq M_2$ or $M_2 \not\subseteq M_1$ ($M_1, M_2$ are in/unequal).

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Operations on sets and relations between sets



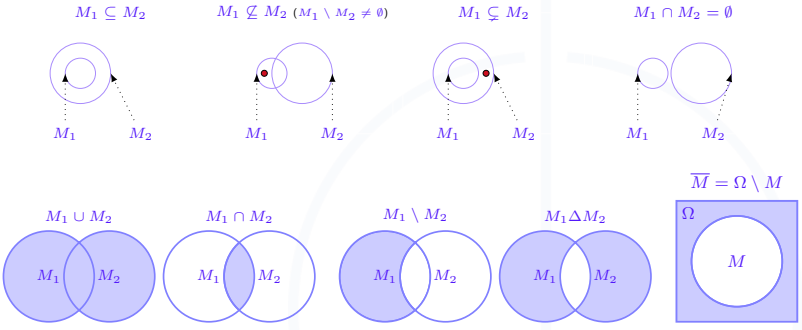

**Def** Let $M_1, M_2$ denote arbitrary (well-defined) sets (Mengen). Then:

- $x \in M_1 \cup M_2$ **:iff** $x \in M_1$ or $x \in M_2$ ($M_1 \cup M_2$ is the union of $M_1, M_2$).
- $x \in M_1 \cap M_2$ **:iff** $x \in M_1$ and $x \in M_2$ ($M_1 \cap M_2$ is the intersection of $M_1, M_2$).
- $x \in M_1 \setminus M_2$ **:iff** $x \in M_1$ and $x \notin M_2$ ($M_1 \setminus M_2$ is the difference of $M_1$ to $M_2$).
- $x \in M_1 \Delta M_2$ **:iff** either $x \in M_1$ or $x \in M_2$ ($M_1 \Delta M_2$ is their symmetric difference).

define again sets. (For a set $M$ only $x \in M$ resp. $x \notin M$ matter.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Operations on sets and relations between sets

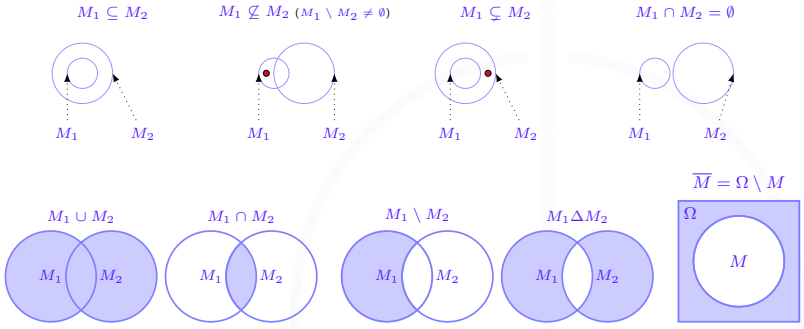



- Intersection, difference and symmetric difference can also be defined using separation and union.
  - $M_1 \setminus M_2 = \{x \in M_1 \mid x \notin M_2\}$
  - $M_1 \Delta M_2 = (M_1 \setminus M_2) \cup (M_2 \setminus M_1) = \{x \in M_1 \cup M_2 \mid x \notin M_1 \text{ or } x \notin M_2\}$.
  - $M_1 \cap M_2 = \{x \in M_1 \mid x \in M_2\} = \{x \in M_2 \mid x \in M_1\} = (M_1 \cup M_2) \setminus (M_1 \Delta M_2)$

  These operations are just "syntactical sugar": they make it simpler to define sets, but are actually not needed.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Operations on sets and relations between sets



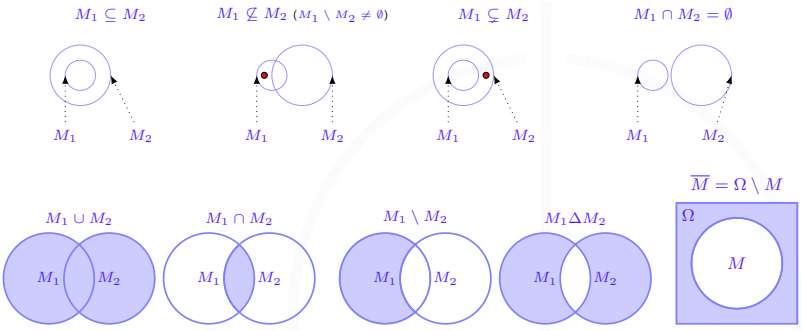

**Def** Derived definitions:

- $M_1 \supseteq M_2$ **:iff** $M_2 \subseteq M_1$ ($M_1$ is a superset of $M_2$).
  (Analogously $M_1 \not\supseteq M_2$ and $M_1 \supsetneq M_2$.)
- $M_1, M_2$ are disjoint **:iff** $M_1 \cap M_2 = \emptyset$.
- If a set $\Omega$ (the "universe", "universal set") is fixed, then $\overline{M} := \Omega \setminus M$ is the complement of $M$ (relative to $\Omega$).

⚠ $\overline{M}$ requires a fixed universe else it is undefined. (There is no set of all sets.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Operations on sets and relations between sets




- Remarks:
  - ⚠ Some authors use $\subset$ in the sense of $\subsetneq$ (cf.: $<, \leq$) others in the sense of $\subseteq$.
  - "$M_1 - M_2$" is also often used for "$M_1 \setminus M_2$".

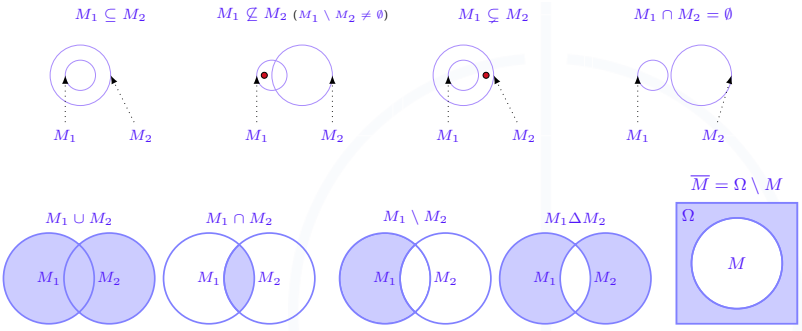
**Fct** $M_1 \subseteq M_2$ **iff** $M_1 \cap M_2 = M_1$ **iff** $M_1 \cup M_2 = M_2$ **iff** $M_1 \setminus M_2 = \emptyset$.

(Cf.: $a \leq b$ **iff** $\min(a, b) = a$ **iff** $\max(a, b) = b$ **iff** $a - b \leq 0$)

**Fct** $M_1 = M_2$ **iff** $M_1 \Delta M_2 = \emptyset$.

For personal use only, no commercial use – TUM CIT Department of Computer Science

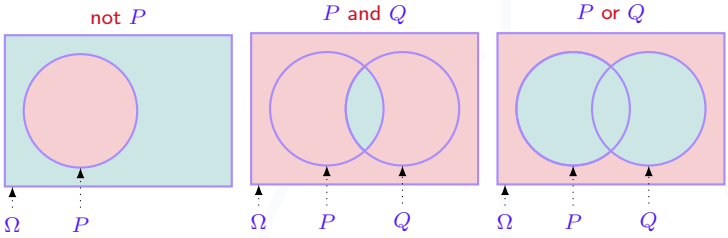
# Operations on sets and relations between sets




**EG** Let $M_1 = \{a, b, c\}$ and $M_2 = \{b, c, d\}$ (with $i = 1, 2$):

- $M_1 = \{b, a, c\} = \{a, a, b, b, c, c\} = M_1 \cup M_1 = M_1 \cap M_1 = \ldots$
- $M_1 \cup M_2 = M_2 \cup M_1 = \{a, b, c, d\}$, $M_i \subsetneq M_1 \cup M_2 \not\subseteq M_i$
- $M_1 \cap M_2 = M_2 \cap M_1 = \{b, c\}$, $M_1 \cap M_2 \subsetneq M_i \not\subseteq M_1 \cap M_2$
- $M_1 \setminus M_2 = \{a\} \neq \{d\} = M_2 \setminus M_1$
- $M_1 \Delta M_2 = M_2 \Delta M_1 = \{a, d\}$

For personal use only, no commercial use – TUM CIT Department of Computer Science



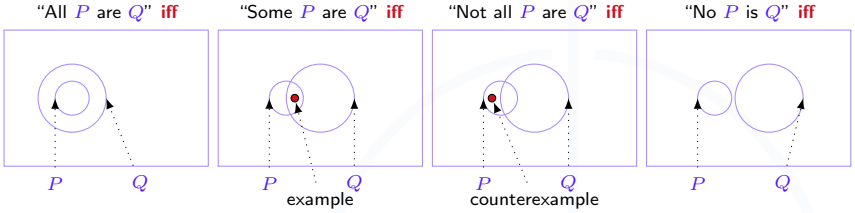

- Fix a nonempty universe $\Omega$ and predicates $P, Q$ that define subsets (no Russell).
  - $\{x \in \Omega \mid \text{not } P(x)\} = \Omega \backslash \{x \in \Omega \mid P(x)\}$
  - $\{x \in \Omega \mid P(x) \text{ and } Q(x)\} = \{x \in \Omega \mid P(x)\} \cap \{x \in \Omega \mid Q(x)\}$
  - $\{x \in \Omega \mid P(x) \text{ or } Q(x)\} = \{x \in \Omega \mid P(x)\} \cup \{x \in \Omega \mid Q(x)\}$

EG Typical example: $\Omega$ is a "database" (external database facts).

$P, Q$ define properties used to "select" specific database entries.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Basic concepts of set theory and logic: comparison




- Fix a nonempty universe $\Omega$ and predicates $P, Q$ that define subsets (no Russell).
  - "All $P$ are $Q$" **iff** $\{x \in \Omega \mid P(x)\} \subseteq \{x \in \Omega \mid Q(x)\}$
    ("Vacuously" true if $\{x \in \Omega \mid P(x)\} = \emptyset$.)
  - "Some $P$ are $Q$" **iff** $\{x \in \Omega \mid P(x)\} \cap \{x \in \Omega \mid Q(x)\} \neq \emptyset$
  - "Not all $P$ are $Q$" **iff** $\{x \in \Omega \mid P(x)\} \cap \{x \in \Omega \mid \text{not } Q(x)\} \neq \emptyset$
    **iff** "Some $P$ are not $Q$"
  - "No $P$ is $Q$" **iff** $\{x \in \Omega \mid P(x)\} \cap \{x \in \Omega \mid Q(x)\} = \emptyset$
    **iff** $\{x \in \Omega \mid P(x)\} \subseteq \{x \in \Omega \mid \text{not } Q(x)\}$
    **iff** "All $P$ are not $Q$"

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Finite sets, cardinality

**Def** A set $M$ is finite **:iff** $M = \{x_1, x_2, \ldots, x_k\}$ for some $k \in \mathbb{N}_0$ and (well-defined) objects $x_1, x_2, \ldots, x_k$.

- ▷ Usual convention: $\{x_1, x_2, \ldots, x_k\} = \emptyset$ for $k = 0$.
- ▷ "infinite" short for "not finite".

**Def** If $M$ is finite, then $|M|$ denotes the natural number of (pairwise) distinct objects that are elements of $M$.

(I.e. $|M|$ is the least natural number $k$ s.t. $M = \{x_1, x_2, \ldots x_k\}$ for suitable objects $x_1, x_2, \ldots, x_k$.)

**EG** $|\{a, b, c\}| = |\{a, b, c, a, b, c\}| = |\{c, a, b, b, c, c, c\}| = |\{a, b\} \cup \{b, c\}| = 3$.

**Def** $|M|$ is called the cardinality of $M$.

- ▷ Often "$|M| < \infty$" and "$|M| = \infty$" are used as abbreviations for "$M$ is finite" and "$M$ is infinite".

⚠ But not all infinite sets have the same cardinality:

The precise definition of the cardinality $|M|$ of an infinite set $M$ requires the definition of injective functions (later).

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Set of sets: Power set

**Def** Let $M$ be set. Then $A \in 2^M$ **:iff** $A \subseteq M$. The so defined set $2^M$ (also $\mathcal{P}(M)$) is called the power set of $M$; its elements are exactly the subsets of $M$.

**EG**

$$
\begin{array}{lcll}
2^{\{1\}} & = & \mathcal{P}(\{1\}) & = \{\emptyset, \{1\}\} \\
2^{\{1,2\}} & = & \mathcal{P}(\{1,2\}) & = \{\emptyset, \{1\}, \{2\}, \{1,2\}\} \\
2^{\{1,2,3\}} & = & \mathcal{P}(\{1,2,3\}) & = \{\emptyset, \{1\}, \{2\}, \{1,2\}\} \\
 & & & \cup \ \{\{3\}, \{1,3\}, \{2,3\}, \{1,2,3\}\} \\
2^{\emptyset} & = & \mathcal{P}(\emptyset) & = \{\emptyset\} \\
2^{\{\emptyset\}} & = & \mathcal{P}(\{\emptyset\}) & = \{\emptyset, \{\emptyset\}\} \\
2^{\{\emptyset,\{\emptyset\}\}} & = & \mathcal{P}(\{\emptyset, \{\emptyset\}\}) & = \{\emptyset, \{\emptyset\}, \{\{\emptyset\}\}, \{\emptyset, \{\emptyset\}\}\}
\end{array}
$$

**Fct** If $x \in M \neq \emptyset$, then $2^M = 2^{M-\{x\}} \cup \{K \cup \{x\} \mid K \in 2^{M-\{x\}}\}$.

**Fct** If $M = \{x_1, \ldots, x_k\}$ is finite with $|M| = k$, then $|2^M| = 2^{|M|} = 2^k$:
$A \subseteq M$ is uniquely determined by $k$ predicates ($x_i \in A$ resp. $x_i \notin A$).

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Sets of sets: Union and intersection in general

**Def** Let $S \neq \emptyset$ be a set whose elements are sets, too. Then:

$$\begin{array}{rcll} \bigcup S & := & \bigcup_{M \in S} M & := & \{x \mid \text{exists } M \in S\colon\ x \in M\} \\ \bigcap S & := & \bigcap_{M \in S} M & := & \{x \mid \text{for all } M \in S\colon\ x \in M\} \end{array}$$

For $S = \{M_1, \ldots, M_k\}$ finite ($k \in \mathbb{N}$) the following notation is used:

$$\bigcup_{i=1}^{k} M_i := \bigcup S \qquad \bigcap_{i=1}^{k} M_i := \bigcap S$$

Further set $\bigcup \emptyset := \emptyset$ (but $\bigcap \emptyset$ is undefined).

- $M_1 \cap M_2 = \bigcap\{M_1, M_2\}$ and $M_1 \cup M_2 = \bigcup\{M_1, M_2\}$
- If $S \neq \emptyset$, then $\bigcap S = \{x \in \bigcup S \mid \text{for all } M \in S\colon\ x \in M\}$.

**EG** $\bigcup 2^M = M$ and $\bigcap 2^M = \emptyset$.

**EG** $\bigcup\big\{\{1,4\},\{1,3,2\},\{1,3\}\big\} = \{1,4\} \cup \{1,3,2\} \cup \{1,3\} = \{1,2,3,4\}$
$\bigcap\big\{\{1,4\},\{1,3,2\},\{1,3\}\big\} = \{1,4\} \cap \{1,3,2\} \cap \{1,3\} = \{1\}$

**EG** $S := \big\{\{x \in \mathbb{R}_{>0}\colon\ \big|2 - x^2\big| < \tfrac{1}{k}\big\} \mid k \in \mathbb{N}\}$; $\bigcup S = (1, \sqrt{3})$; $\bigcap S = \{\sqrt{2}\}$.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Sets in Python



```
A = {0, 1, 2, 3, 4}                 # A := {0,1,2,3,4}
B = {x for x in A if x <= 2}        # B := {x ∈ A | x ≤ 2} = {0,1,2}
C = {x for x in A if x % 2 == 0}    # C := {x ∈ A | x even } = {0,2,4}
D = {2*x for x in A}                # D := {2x | x ∈ A}
E = {frozenset({x}) for x in A}     # E := {{x} | x ∈ A}
F = set()                           # F := {}
for i in range(5): F.add(i)         # for i from 0 to 4: F := F ∪ {i}
print(A,B,C,D,E,F)
```

- In programming languages by means of "list/set comprehensions":
  "`for x in A if P(x)`" corresponds to "$x \in A \mid P(x)$"
  with $P$ a predicate defining a property.
- Remark: "$=$" vs. "$==$"
  - "$=$" in math "in the given context the same".
    E.g. "$5 + 6 = 11$" true after evaluation in $\mathbb{R}$, but false before evaluation.
    In case of definitions: "let $A = \{0,1,2,3,4\}$", or short "$A := \{0,1,2,3,4\}$".
  - "$=$" in Python, C, C++, Java: assignment/definition (let LHS be/denote RHS).
  - "$==$" in Python, C, C++: "equivalent" (as defined/implemented by user).
    ("$==$" in Java: "identical" (primitive or memory address))

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Sets in Python



```
A = {0, 1, 2, 3, 4}                 # A := {0,1,2,3,4}
B = {x for x in A if x <= 2}        # B := {x ∈ A | x ≤ 2} = {0,1,2}
C = {x for x in A if x % 2 == 0}    # C := {x ∈ A | x even } = {0,2,4}
D = {2*x for x in A}                # D := {2x | x ∈ A}
E = {frozenset({x}) for x in A}     # E := {{x} | x ∈ A}
F = set()                           # F := {}
for i in range(5): F.add(i)         # for i from 0 to 4: F := F ∪ {i}
print(A,B,C,D,E,F)
```

- In programming languages by means of "list/set comprehensions":
  "`for x in A if P(x)`" corresponds to "$x \in A \mid P(x)$"
  with $P$ a predicate defining a property.
- Remark: `set` vs. `frozenset`
  - In most programming languages sets are "mutable", e.g. we can start from the empty set and add $0, 1, ..., 4$ one after the other.
  - In math a set is "immutable" after its definition. Immutable sets are called `frozenset`s in Python.
  - A Python `set` cannot be an element of another Python `set`; but a `frozenset` can be an element of a `set` (the definition of the elements of a `set` must not change).

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Sets in Python



```
A = {0,1}; B = {0,2}; C = {0}
print(len(A))                          # |A|
print(A | B, A.union(B))               # A ∪ B
print(A & B, A.intersection(B))        # A ∩ B
print(A - B, A.difference(B))          # A \ B
print(A ^ B, A.symmetric_difference(B)) # AΔB
print(A <= B, A.issubset(B))           # A ⊆ B
print(A <= A, A.issubset(A))           # A ⊆ A
print(A < A)                           # A ⊊ A
print(C < A)                           # C ⊊ A
print(A == {1,0})                      # Order does not matter

def union(S: set[frozenset]): # ⋃{M_1, ..., M_n} = M_1 ∪ ... ∪ M_n
  return set().union(*S)

def intersection(S: set[frozenset]): #⋂{M_1, ..., M_n} = {x ∈ M_1 | x ∈ M_i for all i ≤ n}
  M1, *S = S
  return {x for x in M1 if all(x in M for M in S)}

print(union({frozenset(A),frozenset(B), frozenset(C)}))
print(intersection({frozenset(A),frozenset(B), frozenset(C)}))

def powerset(M: set): # 2^{a_1,a_2,...} = 2^{a_2,...} ∪ {{a_1} ∪ A | A ∈ 2^{a_2,...}}
  if len(M) == 0: return {frozenset()}
  x, *M_x = M
  powM_x = powerset(M_x)
  return powM_x | {K | {x} for K in powM_x}

print(powerset(A))
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

1 Basic concepts

Basic concepts of logic

Sets

Sequences, tuples, and words

The natural numbers, proof by induction

Relations

Functions

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Sets vs. sequences

- Set: collection where neither order nor multiplicities matter:

$$\{a, b, \emptyset, \{b, a\}, a\} = \{\emptyset, \{a, b\}, a, b\}$$

- Sequence: collection where both order and multiplicities matter:

$$(a, b, \emptyset, \{b, a\}, a) \neq (\emptyset, \{a, b\}, a, b)$$

  This time round brackets "(", ")" for definining a sequence.

  Finite sequences are also called tuples.

- ⚠ Many programming languages distinguish lists/arrays from tuples:
  - tuples are in general "immutable (const)", only redefinition possible.
  - lists and arrays are "mutable tuples",

    depending on the programming language further differences (implementation, run time, memory, ...).
  - In geometry and linear algebra tuples are used to represent finite-dimensional vectors, sequences are one example of infinite-dimensional vectors.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Sequences: definition

**Def** Let $I \subseteq \mathbb{N}_0$ ("index set"; typically $I = \mathbb{N}$ or $I = \mathbb{N}_0$ or $I = [n]$ or $I = \mathbb{Z}_n$).
If for every index $i \in I$ an object $a_i$ is defined, then $(a_i)_{i \in I}$ denotes the listing w.r.t. increasing index $i$ which is called a sequence.

- ▷ Alternative notation: $(a_i | i \in I)$ for $(a_i)_{i \in I}$.
- ▷ $a_i$ is called the $i$-th component/entry/member (also: element).
- ▷ $|(a_i)_{i \in I}| := |I|$ is the length of $(a_i)_{i \in I}$; it is finite, if $|I| < \infty$.
- ▷ For $I = [n]$ or $I = \mathbb{Z}_n$ also "tuple of length $n$ ($n$-tuple)" is used.
  Then: $(a_i)_{i \in [n]} = (a_1, \ldots, a_n)$ resp. $(a_i)_{i \in \mathbb{Z}_n} = (a_0, a_1, \ldots, a_{n-1})$
- ▷ Note: $(a_i)_{i \in \emptyset} = ()$ is the empty tuple of length $0$.
- ▷ Tuples can also be represented as nested sets (or as "ordered" functions).

**EG** $\big(i\big)_{i \in \mathbb{N}_0} = \big(0, 1, 2, 3, 4, \ldots\big)$.

**EG** $\big(\frac{1}{i}\big)_{i \in \mathbb{N}} = \big(1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \ldots\big)$.

**Def** $(a_i)_{i \in I} = (b_j)_{j \in J}$ **iff** $I = J$ and $a_i = b_i$ for all $i \in I$.

**EG** $\Big|\Big\{(a,(b,c)), (a,(c,b)), (a,\{b,c\}), (a,\{c,\}), (), (a,b,c)\Big\}\Big| = 5$

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Typical examples of sequences

- Geometric sequence/progession: $a_i := cq^i$ for fixed $c, q \in \mathbb{R}$.
  E.g. for $c = 1$ and $q = -1/2$:

$$1, -\tfrac{1}{2}, \tfrac{1}{4}, -\tfrac{1}{8}, \tfrac{1}{16}, -\tfrac{1}{32}, \ldots$$

  Geometric series/sum: $a_0 := 0$ and $a_{i+1} := a_i + cq^i$ for fixed $c, q \in \mathbb{R}$.
  E.g. for $c = 1$ and $q = -1/2$:

$$0, 1, \tfrac{1}{2}, \tfrac{3}{4}, \tfrac{5}{8}, \tfrac{11}{16}, \tfrac{21}{32}, \ldots$$

  Fibonacci numbers: $F_0 := 0$, $F_1 := 1$, $F_{n+2} := F_{n+1} + F_n$

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots$$

  (Inductive definitions; see Peano axioms later.)
  Approximation of Euler's number $e$: $a_i = (1 + \frac{1}{i})^i$ for $i \in \mathbb{N}$:

$$2, \tfrac{9}{4}, \tfrac{64}{27}, \tfrac{625}{256}, \tfrac{7776}{3125}, \ldots$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Cartesian product and (ordered) pair

**Def** For sets $A, B$ the set $A \times B$ is defined by

$$A \times B := \{(a, b) \mid a \in A, b \in B\}$$

$A \times B$ is called the cartesian product of $A$ and $B$; its elements are exactly those 2-tuples (also: (ordered) pair) whose first and second component are an element of $A$ and $B$, respectively.

| $A \times B$ | $b_1$ | $b_2$ | $b_3$ |
|---|---|---|---|
| $a_1$ | $(a_1, b_1)$ | $(a_1, b_2)$ | $(a_1, b_3)$ |
| $a_2$ | $(a_2, b_1)$ | $(a_2, b_2)$ | $(a_2, b_3)$ |

**Def** Analogously, for a finite sequence $A_1, \ldots, A_k$ of sets:

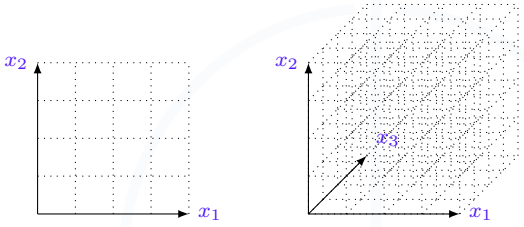
$$A_1 \times \ldots \times A_k := \bigtimes_{i=1}^{k} A_i := \{(a_i)_{i \in [k]} \mid a_i \in A_i \text{ for all } i \in [k]\}$$

Note that for $k = 0$ the definition becomes $\bigtimes_{k=1}^{0} A_k = \{ () \}$.

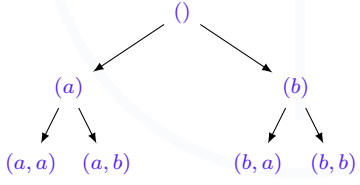
If $A = A_1 = \ldots = A_k$, then simply $A^k$ for $A_1 \times \ldots \times A_k$.

For personal use only, no commercial use – TUM CIT Department of Computer Science

- $\mathbb{R}^2$ and $\mathbb{R}^3$ are often used as "coordinate vector spaces":




- For a fixed finite set $A$, the cartesian products $A^0$, $A^1$, $A^2$, ... can be visualized as the "levels of a rooted tree". E.g. for $A = \{a, b\}$:




(Binary left/right decsions)

For personal use only, no commercial use TUM CIT Department of Computer Science

# Sequences in Python



```
e = ()                                  # empty sequence/tuple
a = tuple(i for i in range(5))          # a = (0,1,2,3,4) = (i)_{i∈Z_5}
b = (e,)                                # (()) (Python needs the trail ',')
w = 'ABCD'                              # words/strings are sequences/tuples of letters
for s in (e,a,b,w):                     # sequences can be elements of sequences
  print(f"s={s}, |s|={len(s)}")
  for i in range(len(s)):               # entry at i, prefix of length i, suffix starting at i
    print(f"  s[{i}]={s[i]}, s[:{i}]={s[:i]}, s[{i}:]={s[i:]}")
  for i in range(1,len(s)+1):           # a[-1]:=a[len(a)-1], a[-2]:=a[len(a)-2], ...
    print(f"  s[{-i}]={s[-i]}")

# Examples from slides
S = (('a',('b','c')), ('a',('c','b')), ('a','c','b'), ('a',frozenset({'b','c'})), ('a',frozenset({'c','b'})))
for i in range(len(S)):
  for j in range(i+1, len(S)):
    print(f"i={i}, j={j}, {S[i]} == {S[j]}? {S[i] == S[j]}")

geo_seq = tuple(pow(-0.5,i) for i in range(12))
geo_sum = tuple(sum(geo_seq[:i]) for i in range(len(geo_seq)+1))
print(f"Geometric sequence: {geo_seq}")
print(f"Geometric sum: {geo_sum}")
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Lists (mutable sequences) in Python



```
e = []                                  # empty list (mutable sequence)
a = [i for i in range(5)]               # a = (0,1,2,3,4) = (i)_{i∈Z_5}
b = list('abcd')                        # convert word to list
c = [e]                                 # The list of length one with the empty list as its only entry
d = []

geo = [1 for _ in range(12)]
for i in range(1,len(geo)):
  geo[i] = geo[i-1] * (-0.5)            # In lists we can also redefine the entry at position/index i
print(f"Geometric sequence: {geo}")
for i in range(1,len(geo)):
  geo[i] += geo[i-1]
print(geo)
print(f"Geometric sum: {geo}")

fib = [0,1]
for i in range(10):
  fib.append(fib[-1] + fib[-2])         # Lists are mutable like sets: we can append elements at their end
print(f"Fibonacci numbers: {fib}")
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Cartesian product in Python



```
from itertools import product          # let Python look up what the Cartesian product is
A = {0,1}; B = {2,3}; C = {4,5}
AxB = {(a,b) for a,b in product(A,B)} # shorter: `set(product(A,B))`
print(f"A x B = {AxB}, |A x B|={len(AxB)}")
# (A × B) × C ≠ A × B × C ≠ A × (B × C)
print(f"(A x B) x C = {set(product(AxB,C))}")
print(f"A x (B x C) = {set(product(A,product(B,C)))}")
print(f"A x B x C   = {set(product(A,B,C))}")
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

1 Basic concepts

Basic concepts of logic

Sets

Sequences, tuples, and words

The natural numbers, proof by induction

Relations

Functions

For personal use only, no commercial use – TUM CIT Department of Computer Science

# The natural numbers

- The natural numbers $\mathbb{N}_0$ (incl. $0$) are the first infinite set:
  - $\mathbb{N}_0$ is the very concept of "countable/enumerable/denumerable/discrete".
  - $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ and $\mathbb{C}$ are based on $\mathbb{N}_0$.
- What is a precise (as necessary) definition of $\mathbb{N}_0$?
  The actual representation of $\mathbb{N}_0$ should be unimportant.
  - $\{_, \square, \square\square, \square\square\square, \square\square\square\square, \square\square\square\square\square, \ldots\}$ unary representation using bricks
  - $\{\varepsilon, |, ||, |||, ||||, |||||, \ldots\}$ unary representation as on a tally stick ("$\varepsilon$" for empty).
  - $\{, \overline{\alpha}, \overline{\beta}, \overline{\gamma}, \overline{\delta}, \overline{\varepsilon}\}$ notation used by Diophantos.
  - $\{0, 1, \ldots, 9, 10, \ldots\} = \mathbb{N}_0$ canoncial decimal representation using arabic numerals
  - $\{0, 1, 01, 11, 001, \ldots\}$ LSBF-binary representation
    $\{0, 1, 10, 11, 100, \ldots\}$ MSBF-binary representation
    $\{0, 1, 11, 10, 110, \ldots\}$ Gray Code
  - $\{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}, \ldots\}$ John von Neumann's representation (axiom of infinity)

(See also: endianness, long/short scale, Hindu-arabic numerals, Liber Abaci by Fibonacci. )

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Peano axioms: informally

- Characterization of the natural numbers by Peano:
  1. There is a distinguished natural number called nil (nihil, none, null, naught, nothing, zero ...).
  2. Every natural number has a uniquely determined successor ("plus one", "one more", "next") which is also a natural number.
  3. For every natural number its successor is different from nil.
  4. If two natural numbers have the same successor, then they are equal.
  5. If a set $X$ contains both nil and with every natural number (in $X$) also its successor, then $X$ contains already all natural numbers. (axiom of induction)
- Essentially, the Peano axioms describe a (unique!) "one-sided infinite path that never goes in circles" ("the nonnegative number line"):




(3 : "no lassos", 4 : "no confluence", 5 : "no second independent number line".)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Axiom of induction: consequences

- ⚠ W.r.t. the Peano axioms for every set $X$: (using canonical $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$)

  $$\mathbb{N}_0 \subseteq X \text{ iff } (\mathrm{IB})\ 0 \in X \text{ and } (\mathrm{IS})\ \text{for all } n \in X \cap \mathbb{N}_0 \text{ also } n+1 \in X.$$

- ▷ The validity is w.r.t. the Peano axioms, i.e. by definition of $\mathbb{N}_0$.
- ▷ Assuming the Peaon axioms we have the rule of inference:

  $$\frac{\begin{array}{ll}(\mathrm{IB}) & 0 \in X \\ (\mathrm{IS}) & \text{for all } n \in X \cap \mathbb{N}_0 \text{ also } n+1 \in X\end{array}}{\mathbb{N}_0 \subseteq X}$$

- ▷ Negation:

  $$\mathbb{N}_0 \not\subseteq X \text{ iff } 0 \notin X \text{ or there is some } n \in X \cap \mathbb{N}_0 \text{ with } n+1 \notin X.$$

  Simplified (requires a formal proof; see well-ordering of $\mathbb{N}_0$):

  $$\mathbb{N}_0 \not\subseteq X \text{ iff there is a least } n_0 \in \mathbb{N}_0 \text{ s.t. } n_0 \notin X.$$

  $n_0$ is then the least counterexample, i.e. for all $n \in \mathbb{N}_0$ with $n < n_0$ we still have $n \in X$.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Axiom of induction: consequences

⚠ W.r.t. the Peano axioms for every set $X$: (using canonical $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$)

$$\mathbb{N}_0 \subseteq X \text{ iff } \text{IB}\ \ 0 \in X \text{ and } \text{IS}\ \text{ for all } n \in X \cap \mathbb{N}_0 \text{ also } n + 1 \in X.$$

▷ Inductive (recursive) definitions: To define a sequence $(a_n)_{n \in \mathbb{N}_0}$ define

- IB explicity initial objects $a_0, \ldots, a_l$ ($l \in \mathbb{N}_0$) and
- IS for all $n \in \mathbb{N}_0$ how the next element of the sequence $a_{l+n+1}$ is uniquely obtained (constructed, computed) from the so-far defined elements $a_0, \ldots, a_{l+n}$.
  - ⚠ The important point is that the definition of $a_{l+n+1}$ must only refer to already defined objects (no Russell).

($X$ is the set of indices $l + n$ for which $a_{l+n}$ is well defined.)

EG For every $n \in \mathbb{N}_0$ we define the number $a_n \in \mathbb{N}_0$ by means of

$$a_n := \begin{cases} 0 & \text{if } n = 0 \\ 2a_{n-1} + 1 & \text{if } n > 0 \end{cases}$$

e.g.

$$a_0 = 0 \quad a_1 = 1 \quad a_2 = 3 \quad a_3 = 7 \quad a_4 = 15 \quad \ldots$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Axiom of induction: consequences

⚠ W.r.t. the Peano axioms for every set $X$: (using canonical $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$)

$$\mathbb{N}_0 \subseteq X \text{ iff } \text{IB } 0 \in X \text{ and } \text{IS } \text{for all } n \in X \cap \mathbb{N}_0 \text{ also } n + 1 \in X.$$

▷ Proof by induction: Let $P$ be a predicate defined for all $n \in \mathbb{N}_0$.
To show that $P(n)$ is true for all $n \in \mathbb{N}_0$ show

- IB (directly) that $P(0)$ is true (must not refer to any other $P(n)$) and
- IS whenever $P(n)$ is true for an arbitrary (but fixed) $n \in \mathbb{N}_0$ also $P(n + 1)$ is true.

($X = \{n \in \mathbb{N}_0 \mid P(n)\}$ is the set of all $n \in \mathbb{N}_0$ for which $P$ is true.)

EG For every $n \in \mathbb{N}_0$ we define the number $a_n \in \mathbb{N}_0$ by means of

$$a_n := \begin{cases} 0 & \text{if } n = 0 \\ 2a_{n-1} + 1 & \text{if } n > 0 \end{cases}$$

Then: $a_n = 2^n - 1$ for all $n \in \mathbb{N}_0$ (i.e. with $P(n)$ the predicate "$a_n = 2^n - 1$"; $a_n$ is well-defined).

For personal use only, no commercial use – TUM CIT Department of Computer Science

EG For every $n \in \mathbb{N}_0$ we define the number $a_n \in \mathbb{N}_0$ by means of

$$a_n := \begin{cases} 0 & \text{if } n = 0 \\ 2a_{n-1} + 1 & \text{if } n > 0 \end{cases}$$

e.g.

$$a_0 = 0 \quad a_1 = 1 \quad a_2 = 3 \quad a_3 = 7 \quad a_4 = 15 \quad \ldots$$

- Conjecture: $a_n = 2^n - 1$ for all $n \in \mathbb{N}_0$, i.e. $\{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\} = \mathbb{N}_0$.
- We show that $X := \{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\}$ satisfies:
  - IB $0 \in X$ (induction basis/base, base/initial case)
  - IS For every $n \in X \cap \mathbb{N}_0$ also $n + 1 \in X$ holds. (induction step)

For personal use only, no commercial use – TUM CIT Department of Computer Science

EG For every $n \in \mathbb{N}_0$ we define the number $a_n \in \mathbb{N}_0$ by means of

$$a_n := \begin{cases} 0 & \text{if } n = 0 \\ 2a_{n-1} + 1 & \text{if } n > 0 \end{cases}$$

e.g.

$$a_0 = 0 \quad a_1 = 1 \quad a_2 = 3 \quad a_3 = 7 \quad a_4 = 15 \quad \dots$$

- Conjecture: $a_n = 2^n - 1$ for all $n \in \mathbb{N}_0$, i.e. $\{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\} = \mathbb{N}_0$.

IB Induction basis: We show $0 \in X$.

I.e. we have to show that $a_0 = 2^0 - 1$.

- Left-hand side (LHS): $a_0 = 0$ by definition.
- Right-hand side (RHS): $2^0 - 1 = 1 - 1 = 0$.

For personal use only, no commercial use – TUM CIT Department of Computer Science

EG For every $n \in \mathbb{N}_0$ we define the number $a_n \in \mathbb{N}_0$ by means of

$$a_n := \begin{cases} 0 & \text{if } n = 0 \\ 2a_{n-1} + 1 & \text{if } n > 0 \end{cases}$$

e.g.

$$a_0 = 0 \quad a_1 = 1 \quad a_2 = 3 \quad a_3 = 7 \quad a_4 = 15 \quad \dots$$

- ▷ Conjecture: $a_n = 2^n - 1$ for all $n \in \mathbb{N}_0$, i.e. $\{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\} = \mathbb{N}_0$.

IS Induction step: We show that for every $n \in X \cap \mathbb{N}_0$ also $n + 1 \in X$ holds.

Fix any arbitrary $n \in \mathbb{N}_0$.

- ▷ Induction assumption: $n \in X$, i.e. we assume $a_n = 2^n - 1$ holds.

  (Often also called induction hypothesis.)
- ▷ Induction claim: $n + 1 \in X$, i.e. we have to prove $a_{n+1} = 2^{n+1} - 1$.
- ▷ Proof of the induction claim:

$$a_{n+1} \overset{\text{Def.}}{=} 2a_n + 1 \overset{\text{I.A.}}{=} 2(2^n - 1) + 1 = 2^{n+1} - 2 + 1 = 2^{n+1} - 1$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

EG For every $n \in \mathbb{N}_0$ we define the number $a_n \in \mathbb{N}_0$ by means of

$$a_n := \begin{cases} 0 & \text{if } n = 0 \\ 2a_{n-1} + 1 & \text{if } n > 0 \end{cases}$$

e.g.

$$a_0 = 0 \quad a_1 = 1 \quad a_2 = 3 \quad a_3 = 7 \quad a_4 = 15 \quad \ldots$$

- Conjecture: $a_n = 2^n - 1$ for all $n \in \mathbb{N}_0$, i.e. $\{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\} = \mathbb{N}_0$.
- We have shown that $X := \{n \in \mathbb{N}_0 \mid a_n = 2^n - 1\}$ satisfies:
  - IB $0 \in X$ (induction basis/base, base/initial case) ✓
  - IS For every $n \in X \cap \mathbb{N}_0$ also $n + 1 \in X$ holds. (induction/inductive step) ✓
- By virtue of the axiom of induction: $\mathbb{N}_0 \subseteq X$, and thus $X = \mathbb{N}_0$.
- Hence, $a_n = 2^n - 1$ for all $n \in \mathbb{N}_0$.
- Proofs by induction everywhere, e.g. for correctness of recursive programs or programs with while-loops. (See "functional programming and verification".)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Tower of Hanoi

```
def toh(n: int, src: str="L", dst: str="R", aux: str="M"):
    if n > 0:
        toh(n-1,src, aux, dst)
        print(f"move top disk from {src} to {dst}")
        toh(n-1,aux, dst, src)
    return
```




- Number of moves (requires actually also a proof by induction on the program):

$$a_0 = 0 \qquad a_{n+1} = 2a_n + 1$$

As just shown: $a_n = 2^n - 1$. So, for $n = 3$ disks $7$ moves.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Tower of Hanoi

```
def toh(n: int, src: str="L", dst: str="R", aux: str="M"):
    if n > 0:
        toh(n-1,src, aux, dst)
        print(f"move top disk from {src} to {dst}")
        toh(n-1,aux, dst, src)
    return
```




- Number of moves (requires actually also a proof by induction on the program):

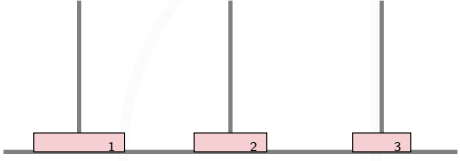
$$a_0 = 0 \qquad a_{n+1} = 2a_n + 1$$

As just shown: $a_n = 2^n - 1$. So, for $n = 3$ disks $7$ moves.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Tower of Hanoi

```
def toh(n: int, src: str="L", dst: str="R", aux: str="M"):
    if n > 0:
        toh(n-1,src, aux, dst)
        print(f"move top disk from {src} to {dst}")
        toh(n-1,aux, dst, src)
    return
```




- Number of moves (requires actually also a proof by induction on the program):

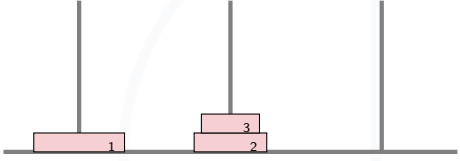
$$a_0 = 0 \qquad a_{n+1} = 2a_n + 1$$

As just shown: $a_n = 2^n - 1$. So, for $n = 3$ disks $7$ moves.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Tower of Hanoi

```
def toh(n: int, src: str="L", dst: str="R", aux: str="M"):
    if n > 0:
        toh(n-1,src, aux, dst)
        print(f"move top disk from {src} to {dst}")
        toh(n-1,aux, dst, src)
    return
```




- Number of moves (requires actually also a proof by induction on the program):

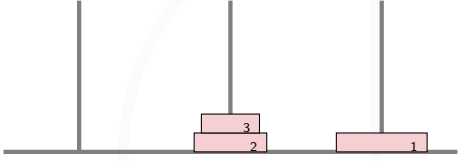
$$a_0 = 0 \qquad a_{n+1} = 2a_n + 1$$

As just shown: $a_n = 2^n - 1$. So, for $n = 3$ disks $7$ moves.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Tower of Hanoi

```
def toh(n: int, src: str="L", dst: str="R", aux: str="M"):
    if n > 0:
        toh(n-1,src, aux, dst)
        print(f"move top disk from {src} to {dst}")
        toh(n-1,aux, dst, src)
    return
```




- Number of moves (requires actually also a proof by induction on the program):

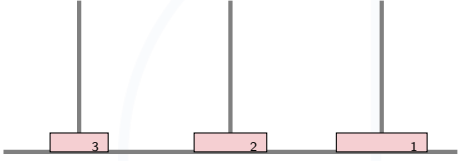
$$a_0 = 0 \qquad a_{n+1} = 2a_n + 1$$

As just shown: $a_n = 2^n - 1$. So, for $n = 3$ disks $7$ moves.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Tower of Hanoi

```
def toh(n: int, src: str="L", dst: str="R", aux: str="M"):
    if n > 0:
        toh(n-1,src, aux, dst)
        print(f"move top disk from {src} to {dst}")
        toh(n-1,aux, dst, src)
    return
```




- Number of moves (requires actually also a proof by induction on the program):

$$a_0 = 0 \qquad a_{n+1} = 2a_n + 1$$

As just shown: $a_n = 2^n - 1$. So, for $n = 3$ disks $7$ moves.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Tower of Hanoi

```
def toh(n: int, src: str="L", dst: str="R", aux: str="M"):
    if n > 0:
        toh(n-1,src, aux, dst)
        print(f"move top disk from {src} to {dst}")
        toh(n-1,aux, dst, src)
    return
```




- Number of moves (requires actually also a proof by induction on the program):

$$a_0 = 0 \qquad a_{n+1} = 2a_n + 1$$

As just shown: $a_n = 2^n - 1$. So, for $n = 3$ disks $7$ moves.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Tower of Hanoi

```
def toh(n: int, src: str="L", dst: str="R", aux: str="M"):
    if n > 0:
        toh(n-1,src, aux, dst)
        print(f"move top disk from {src} to {dst}")
        toh(n-1,aux, dst, src)
    return
```




- Number of moves (requires actually also a proof by induction on the program):

$$a_0 = 0 \qquad a_{n+1} = 2a_n + 1$$

As just shown: $a_n = 2^n - 1$. So, for $n = 3$ disks $7$ moves.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Tower of Hanoi

```
def toh(n: int, src: str="L", dst: str="R", aux: str="M"):
    if n > 0:
        toh(n-1,src, aux, dst)
        print(f"move top disk from {src} to {dst}")
        toh(n-1,aux, dst, src)
    return
```




- Number of moves (requires actually also a proof by induction on the program):

$$a_0 = 0 \qquad a_{n+1} = 2a_n + 1$$

  As just shown: $a_n = 2^n - 1$. So, for $n = 3$ disks $7$ moves.

- For $n = 7$ disks already $127$ moves.

  (Show that this is also the minimal number of moves.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Structure of a proof by induction

- To show "for all $n \in \mathbb{N}_0$: $P(n)$" using induction wrt. $n$:
  - Induction basis (I.B.): Prove $P(0)$ (wrt. the already made assumptions).
  - Induction step (I.S.): Fix an arbitrary $n \in \mathbb{N}_0$ unrestricted by any assumption.
    - Induction assumption (I.A.):
      $P(n)$ holds for the fixed $n$ (wrt. the already made assumptions).
    - Induction claim (I.C.):
      $P(n+1)$ holds for fixed $n$ (wrt. the already made assumptions and $P(n)$).
    - Then actually prove $P(n+1)$ based on the already made assumptions and in addition the induction asssumption $P(n)$ on the fixed $n$.
- Some variants:
  - "For all $n \in \mathbb{N}_0$ with $n \geq n_0$: $P(n)$." $(n_0 \in \mathbb{N}_0)$
    I.B.: Show $P(n_0)$; I.S.: Fix an aribtrary $n \geq n_0$.
  - Strong induction:
    I.A.: $P(k)$ holds for all $n_0 \leq k \leq n$; I.C.: $P(n+1)$ also holds. $(n_0, k \in \mathbb{N})$
    (I.e. replace $P(n)$ by "$P'(n) :=$ for all $n_0 \leq k \leq n$: $P(k)$ is true".)

(Visualize by means of number line/growing subsets.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Example: geometric sum, finite sums and products

- E.g. geometric sum (or "finite geometric series"; appears everywhere)
  For all $n \in \mathbb{N}_0$ and for all $x \in \mathbb{R}$ (also $\mathbb{C}$) with $x \neq 1$:

$$\sum_{k=0}^{n} x^k := 1 + x + \ldots + x^n = \frac{1 - x^{n+1}}{1 - x}$$

- ▷ Proof by induction on $n \in \mathbb{N}_0$:

  I.B.: For $n = 0$ we have on the LHS $\sum_{k=0}^{0} x^k = 1$ for all $x \in \mathbb{R}$.

  On the RHS $\frac{1-x^{0+1}}{1-x} = 1$ for all $x \in \mathbb{R} \setminus \{1\}$.

  I.S.: Fix $n \in \mathbb{N}_0$ aribtrarily.

  I.A.: For the now fixed $n$ we have that $\sum_{k=0}^{n} x^k = \frac{1-x^{n+1}}{1-x}$ for all $x \in \mathbb{R} \setminus \{1\}$.

  I.C.: For the now fixed $n$ we have that $\sum_{k=0}^{n+1} x^k = \frac{1-x^{n+2}}{1-x}$ for all $x \in \mathbb{R} \setminus \{1\}$.

  Proof: For every specific $x \in \mathbb{R} \setminus \{1\}$:

$$\text{(LHS)} \sum_{k=0}^{n+1} x^k = x^{n+1} + \sum_{k=0}^{n} x^k \overset{\text{IA}}{=} x^{n+1} + \frac{1 - x^{n+1}}{1 - x} = \frac{1 - x^{n+2}}{1 - x} \text{ (RHS)}$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Example: geometric sum, finite sums and products

⚠ E.g. geometric sum (or "finite geometric series"; appears everywhere)

For all $n \in \mathbb{N}_0$ and for all $x \in \mathbb{R}$ (also $\mathbb{C}$) with $x \neq 1$:

$$\sum_{k=0}^{n} x^k := 1 + x + \ldots + x^n = \frac{1 - x^{n+1}}{1 - x}$$

**Def** For a real sequence $(a_i)_{i \in \mathbb{N}_0}$ define $\sum_{i=0}^{n} a_i$ inductively:

I.B.: If $n = 0$: $\sum_{i=0}^{n} a_i := a_0$.

I.S.: If $n > 0$: $\sum_{i=0}^{n} a_i := a_n + \sum_{i=0}^{n-1} a_i$.

(Note that the sum is to be computed by adding up the $a_i$'s w.r.t. increasing $i$.)

Analogously for $(a_i)_{i \in \mathbb{N}}$ with $\sum_{i=1}^{0} a_i := 0$.

(The choice of the summation index is unconstrained as long as not already used.)

Similarly, $\prod_{j=1}^{n} a_j$ is defined inductively by:

$\prod_{j=1}^{0} a_j := 1$ and $\prod_{j=1}^{n+1} a_j := a_{n+1} \cdot \prod_{j=1}^{n} a_j$.

▷ Also $\bigcup_{i=1}^{n} A_i, \bigcap_{i=1}^{n} A_i, A_1 \times \ldots \times A_n, \Sigma^*, \ldots$ are inductive definitions.

And both $|2^A| = 2^{|A|}$ and $|A_1 \times \ldots \times A_n| = |A_1| \cdots |A_n|$ can be proven inductively for finite sets.

For personal use only | Commercial use ... FAU CIT Department of Computer Science

```python
n_max = 10
fib = [0,1]
for i in range(n_max+1):
  # extend the sequence inductively
  fib.append(fib[-1] + fib[-2])
print(f"Fibonacci numbers: {fib}")

sum_fib = [fib[0]]
# Colab might read `sum_fib` and (try to) complete the rest of the code
for i in range(1,len(fib)):
  # extend the sequence inductively
  sum_fib.append(sum_fib[-1] + fib[i])
print(f"Sum of Fibonacci numbers: {sum_fib}")

# Compare the two sequences.
# How might `fib` and `sum_fib` be related?
# (Drop the first two entries of `fib`, i.e. consider `fib[2:]`)
# Guess the general rule from these examples (inductive reasoning).
# Then prove your conjecture.
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

1 Basic concepts
Basic concepts of logic
Sets
Sequences, tuples, and words
The natural numbers, proof by induction
Relations
Binary relations
Functions

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Binary relations

Def $R \subseteq A \times B$ (with $A \neq \emptyset \neq B$) is a binary relation.

- In the case of $A = B$ also "homogeneous relation" or "endorelation".
- $aR := \{b \in B \mid (a,b) \in R\}$: successors of $a \in A$ wrt. $R$.
  $Rb := \{a \in A \mid (a,b) \in R\}$: predecessors of $b \in B$ wrt. $R$.

- Examples:
  - "parent-of", "father-of", "child-of", "descendant-of", "ancestor-of"
    "depends-on", "required-for", "sufficient-for", "necessary-for"
    "input-output", "points-at"
    $\{(x,y) \in A \times B \mid P(x,y)\}$ for a binary predicate $P$.
  - Basic concept underlying also data structures like lists and trees.
  - Standard relations on $\mathbb{R}$ (and $\mathbb{N}_0, \mathbb{Z}, \mathbb{Q}$):
    "less-than-or-equal-to" $\leq$, "(strictly) less-than" $<$, "equal" $=$, "unequal" $\neq$
  - ⚠ Notational problem: $= \; = \{(x,x) \mid x \in \mathbb{R}\} \subseteq \mathbb{R} \times \mathbb{R}$ (?!)
    Thus e.g. $<_{\mathbb{R}}$ for "less-than on $\mathbb{R}$", $=_{\mathbb{N}}$ and $\mathrm{id}_{\mathbb{N}}$ for "equal on $\mathbb{N}$" etc.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Binary relations

**Def** $R \subseteq A \times B$ (with $A \neq \emptyset \neq B$) is a binary relation.

- ▷ In the case of $A = B$ also "homogeneous relation" or "endorelation".
- ▷ $aR := \{b \in B \mid (a,b) \in R\}$: successors of $a \in A$ wrt. $R$.
  $Rb := \{a \in A \mid (a,b) \in R\}$: predecessors of $b \in B$ wrt. $R$.

**Def** Identity (relation) on a set $A$: $\mathrm{id}_A := \{(a,a) \mid a \in A\}$

**Def** Infix notation: "$aRb$" for "$(a,b) \in R$"; and $a \not R\, b$ for "$(a,b) \notin R$".

- ▷ e.g. $3 \leq_{\mathbb{N}} 5$ for $(3,5) \in \leq_{\mathbb{N}}$

**Def** Transposed (converse, inverse, dual) relation $R^\top := R^{-1} := \{(b,a) \mid (a,b) \in R\}$.

- ▷ e.g. $\leq_{\mathbb{R}}^\top = \geq_{\mathbb{R}}$, "sufficient-for$^\top$ = necessary-for"

**Def** For later: Let $R \subseteq A \times B$ be a binary relation. $R$ is a ...

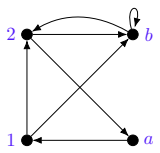
... partial function **:iff** every $a \in A$ has at most one successor ($|aR| \leq 1$).

... (total) function **:iff** every $a \in A$ has a unique successor ($|aR| = 1$).

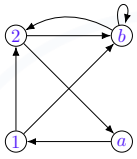
For personal use only, no commercial use – TUM CIT Department of Computer Science

# Binary relations as graphs

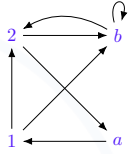
EG $E = \{(1,b),(2,b),(2,a),(1,2),(b,b),(b,2),(a,1)\}$ with $V = \{1,2,a,b\}$




or




or



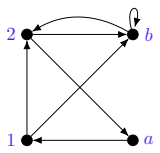

- Visualization of binary relations as directed graphs.
- Formal definition of graph later in DS, for now:
  Given a (finite) relation $E \subseteq A \times B$ let $V = A \cup B$:
  - Elements of $V$ called "nodes" (also "vertex").
  - Elements of $E$ called "edges".
  - For each node $v \in V$ "draw a dot/circle with name $v$".
  - For each edge $(s,t) \in E$ draw an "arrow" from "dot $s$" to "dot $t$".
    If $s = t$, "draw a (self-)loop at $s$". ($s$ forr "source", $t$ for "target".)

(In general, $A, B$ cannot be recover from $A \cup B$ so depending on the application we need to remember $A, B$.)

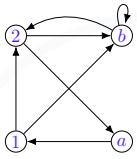
For personal use only, no commercial use – TUM CIT Department of Computer Science

# Binary relations as graphs

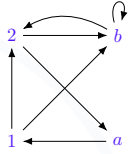
**EG** $E = \{(1,b),(2,b),(2,a),(1,2),(b,b),(b,2),(a,1)\}$ with $V = \{1,2,a,b\}$




or




or




**Def** A sequence $(v_0,\ldots,v_l) \in V^{l+1}$ of nodes is a

(i) path (of length $l$) **:iff** $(v_{i-1},v_i) \in E$ for all $i \in [l]$;

(ii) a cycle **:iff** $(v_0,\ldots,v_l)$ is a path with $v_0 = v_l$ and $l > 0$.

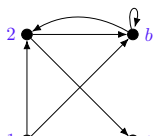
- $v_i$: position at time $i$; in $l$ steps along the edges from $v_0$ to $v_l$.
- Length corresponds to the number of steps/traversed edges.
- Often simply $v_0,\ldots,v_l$ or $v_0 \ldots v_l$, if not ambiguous.

**EG** $(2,a,1,b)$ path of length $3$; $(1,b,2,a,1)$ path (cycle) of length $4$; $(b,b)$ path (cycle) of length $1$; $(1)$ path of length $0$ (no cycle); $(1,b,2,1)$ is not a path.

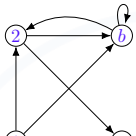
**Def** $s$ can reach $t$ **:iff** there is a path of finite length from $s$ to $t$.

For personal use only, no commercial use – TUM CIT Department of Computer Science

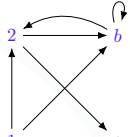
EG $E = \{(1,b),(2,b),(2,a),(1,2),(b,b),(b,2),(a,1)\}$ with $V = \{1,2,a,b\}$




or




or




⚠ Graphs/relations in general do not have a geometric meaning:

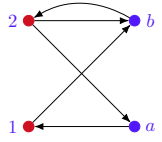
- Positions of the nodes are irrelevant, edges do not need to be straight line segments (see later graph isomorphism). Still try to avoid/minimize overlapping nodes and edges (see planar graphs later).
- Precise representation can vary with application.
- The "visual depiction" of a graph/relation is also referred to as a graphical representation (diagram, diagrammatic form) of the graph/relation. (But often people do not distinguish between the relation/graph and its particular representation.)

- If the nodes can be colored using only two color s.t. every edge only connects nodes of distinct color, then draw all nodes of one color left, the others right (bipartite graph).

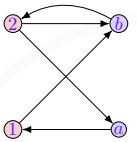
For personal use only, no commercial use – TUM CIT Department of Computer Science

# Binary relations as graphs

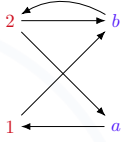
EG $E = \{(1,b),(2,b),(2,a),(b,2),(a,1)\}$ with $V = \{1,2,a,b\}$




or

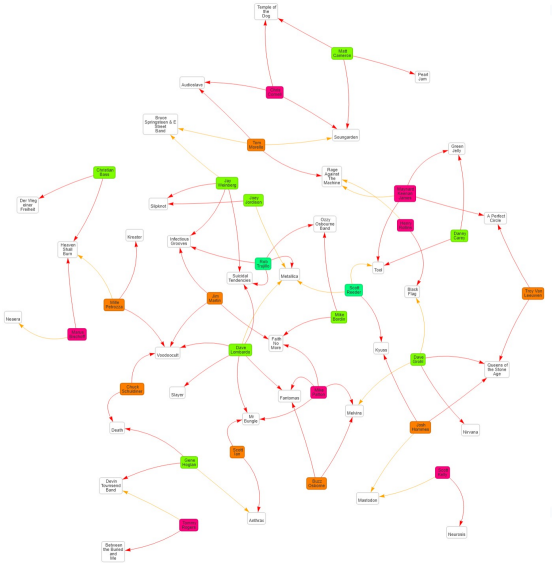



or




- Graphs/relations in general do not have a geometric meaning:
  - Positions of the nodes are irrelevant, edges do not need to be straight line segments (see later graph isomorphism). Still try to avoid/minimize overlapping nodes and edges (see planar graphs later).
  - Precise representation can vary with application.
  - The "visual depiction" of a graph/relation is also referred to as a graphical representation (diagram, diagrammatic form) of the graph/relation. (But often people do not distinguish between the relation/graph and its particular representation.)
- If the nodes can be colored using only two color s.t. every edge only connects nodes of distinct color, then draw all nodes of one color left, the others right (bipartite graph).

For personal use only, no commercial use – TUM CIT Department of Computer Science




$M :=$ "member of" and
$G :=$ "guest appearance" relate persons to bands (binary predicates) and give rise to edge colors.

The graph is bipartite (persons, bands) but the node colors here further "partition" persons by means of their main instrument (unary predicates).

The computer actually does not care about the "meaning/semantics" you give the "words" and "colors", it only "sees" sets (lists, tables) of tuples which it can combine to "infer" new relations like "persons $X, Y$ cooperated" (see composition of relations, transitive closure, (S)CCs, orders, resolution, ... later).

Thus, the computer will also accept any given ("data") list of tuples as "facts" (assume that this was done/happened) and so "base" its "reasoning" on false premisses.

For personal use only, no commercial use – TUM CIT Department of Computer Science

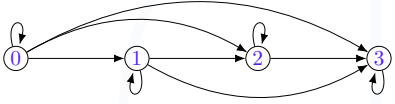
# Less-than-or-equal-to

**Def** $\leq_{\mathbb{N}_0} := \{(a, a+b) \mid a, b \in \mathbb{N}_0\}$

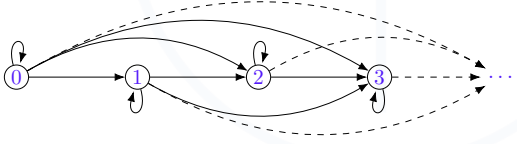
$\leq_{\mathbb{Z}} := \{(a, a+b) \mid a \in \mathbb{Z}, b \in \mathbb{N}_0\}$

$\leq_{\mathbb{Q}} := \{(\frac{a}{b}, \frac{c}{d}) \mid a, c \in \mathbb{Z}, b, d \in \mathbb{N}, ad \leq_{\mathbb{Z}} bc\}$

**EG** $\{(a, b) \in \mathbb{Z}_4^2 \mid a \leq_{\mathbb{N}_0} b\}$ ($\leq_{\mathbb{N}_0}$ restricted to $\mathbb{Z}_4$)



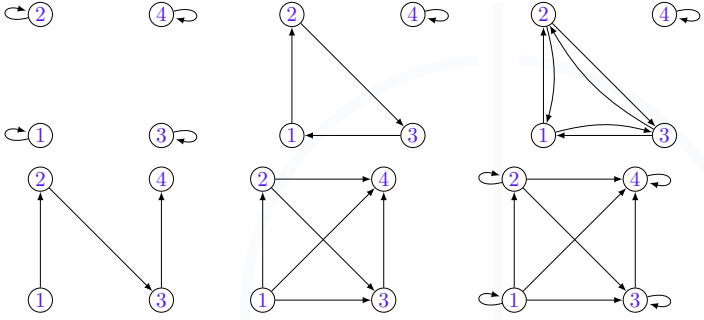

**EG** Sketch of $\leq_{\mathbb{N}_0} \subseteq \mathbb{N}_0 \times \mathbb{N}_0$




(In case of $\mathbb{Q}$ there is no least difference: $\frac{a}{b} \leq \frac{ad+bc}{2bd} \leq \frac{c}{d}$.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Endorelations: properties

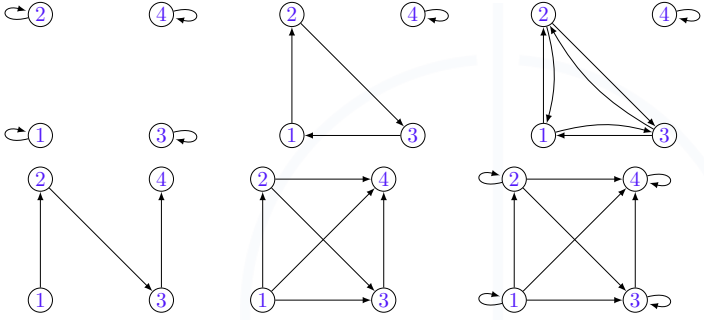



**Def** A binary relation $R \subseteq A \times A$ is called endorelation. $R$ is

- reflexive **:iff** "for all $a \in A$: $(a, a) \in R$" ($\mathrm{id}_A \subseteq R$)
  ("Every node has a loop")
- irreflexive **:iff** "for all $a \in A$: $(a, a) \notin R$" ($\mathrm{id}_A \cap R = \emptyset$)
  ("No node has a loop")

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Endorelations: properties

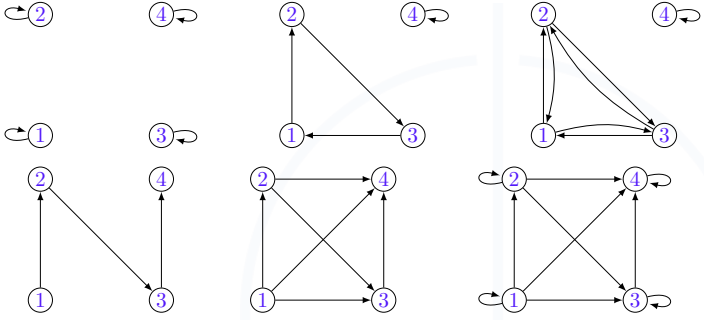



**Def** A binary relation $R \subseteq A \times A$ is called endorelation. $R$ is

- symmetric **:iff** "for all $(s,t) \in R$: $(t,s) \in R$" ($R^\top = R$)
  ("Between any two nodes either both edges or none")
- antisymmetric **:iff** "for all $(s,t) \in R$: if $(t,s) \in R$, then $s = t$" ($R \cap R^\top \subseteq \mathsf{id}_A$)
  ("Between two nodes at most one edge")
- asymmetric, **:iff** "for all $(s,t) \in R$: $(t,s) \notin R$" ($R \cap R^\top = \emptyset$)
  (irreflexive and antisymmetic; cf. $<_{\mathbb{N}_0}$ vs. $\leq_{\mathbb{N}_0}$.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Endorelations: properties




**Def** A binary relation $R \subseteq A \times A$ is called endorelation. $R$ is

- ▷ transitive **:iff** "for all $(s,t),(t,u) \in R$: $(s,u) \in R$"
  (Short: $RR := \{(s,u) \mid (s,t),(t,u) \in R\} \subseteq R$)
  ("Wherever you get in two steps, you can get in one step.")

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Important endorelations

- Examples:
  - $=_{\mathbb{Z}}$: reflexive, symmetric, antisymmetric, transitive
  - $\leq_{\mathbb{Z}}$: reflexive, antisymmetric, transitive
  - $<_{\mathbb{Z}}$: asymmetric (irreflexive and antisymmetric), transitive
  - $\neq_{\mathbb{Z}}$: irreflexive, symmetric, not transitive
  - $|_{\mathbb{Z}} \subseteq \mathbb{Z} \times \mathbb{Z}$ with "$a|b$" defined by "$\frac{b}{a} \in \mathbb{Z}$" (i.e. "$a$ divides $b$")
    only transitive
    Alternatively: "$a|b$" defined by "$b = ka$ for some $k \in \mathbb{Z}$".
    (I.e. "$a$ divides $b$" iff "$b$ is multiple of $a$"; properties now?)
  - $|_{\mathbb{N}} \subseteq \mathbb{N} \times \mathbb{N}$ (restriction of "divides" to the positive integers)
    reflexive, antisymmetric, transitive (both definitions coincide on $\mathbb{N}$)
  - $\equiv_m \subseteq \mathbb{Z} \times \mathbb{Z}$ with "$a \equiv_m b$" defined by "$m|(a-b)$" for a given $m \in \mathbb{N}$
    reflexive, symmetric, transitive
  - $\subseteq$ on $\mathcal{P}(\mathbb{Z})$: reflexive, antisymmetric, transitive
  - Congruence in geometry: reflexive, symmetric, transitive

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Order relations vs. equivalence relations

**Def** $R \subseteq A \times A$ is a (partial) order **:iff** $R$ is reflexive, transitive and antisymmetric.

**EG** $\leq_{\mathbb{Z}}, |_{\mathbb{N}}, \subseteq$.

- ▷ "$a$ comes before $b$" in given context e.g. for sorting data w.r.t. first name or last name or age ("(sort) key").

**Def** $R \subseteq A \times A$ is an equivalence relation **:iff** $R$ is refl., transitive and symmetric.

**EG** $\mathrm{id}_A$, $=_{\mathbb{Z}}$, $\equiv_m$, $=_{\mathbb{Q}}$.

- ▷ "$a$ and $b$ are of equal value (equal, equivalent, interchangeable, similar)" in a given context e.g. for merging contacts. (See also phenotype, substitution property.)
- ▷ "Identical" means "indistinguishable" (see $\mathrm{id}_A$, Leibniz's law: [1], [2], [3], [4]).
- ▷ "Equality" in mathematics amounts to "equal value" e.g. $1+5$ and $2+3$ are different expressions for the same value (see balance scale, transforming equations).
- ▷ "Identity" in mathematics can also refer to a stronger notion of equality when two expressions (involving indeterminants) always evaluate to the same value (in a given context) e.g. $(x+y)^2 = x^2 + 2xy + y^2$ is true for all $x, y \in \mathbb{R}$ ("$\equiv_{\mathbb{R}}$").
- ▷ Related equivalence relations: "**iff**", "**syn**" (see synonym, semantic relations), congruence relations $\equiv$ on expressions (words, terms, formulas).

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
import networkx as nx
import matplotlib.pyplot as plt
from itertools import product
from typing import *

def plot_rel(R: Set[Tuple[Any,Any]], Universe: Optional[Set] = None, pos=None):
  print(R)
  G = nx.DiGraph()
  if Universe is not None:
    G.add_nodes_from(Universe)
  G.add_edges_from(R)
  if pos is None:  # assign positions if necessary
    pos=nx.kamada_kawai_layout(G, pos=nx.spring_layout(G, seed=42))
  nx.draw(G, with_labels=True, pos=pos)
  plt.show()

# Examples from slides
Omega = {1,2,3,4}
ID = {(1,1),(2,2),(3,3),(4,4)}
R1 = {(1,2),(2,3),(3,1), (4,4)}
R2 = R1 | {(1,3),(3,2),(2,1)}
R3 = {(1,2),(2,3),(3,4)}
R4 = R3 | {(1,3),(1,4),(2,4)}
R5 = R4 | {(1,1),(2,2),(3,3),(4,4)}

pos = {1: (0,0), 2: (0,1), 3: (1,0), 4: (1,1)}  # positions as in slides
for R in (ID,R1,R2,R3,R4,R5):
  plot_rel(R,Omega,pos=pos)
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

```
from itertools import product

is_reflexive = lambda R, Universe: all( (a,a) in R for a in Universe)
is_irreflexive = lambda R, Universe: all( (a,a) not in R for a in Universe)
is_symmetric = lambda R: all( (b,a) in R for (a,b) in R)
is_antisymmetric = lambda R: all( (b,a) not in R for (a,b) in R if a != b)
is_asymmetric = lambda R: all( (b,a) not in R for (a,b) in R)
is_transitive = lambda R: all( (a,c) in R for (a,b1),(b2,c) in product(R, repeat=2) if b1==b2)

# Examples from slides
Omega = {1,2,3,4}
ID = {(1,1),(2,2),(3,3),(4,4)}
R1 = {(1,2),(2,3),(3,1), (4,4)}
R2 = R1 | {(1,3),(3,2),(2,1)}
R3 = {(1,2),(2,3),(3,4)}
R4 = R3 | {(1,3),(1,4),(2,4)}
R5 = R4 | {(1,1),(2,2),(3,3),(4,4)}

for R in (ID,R1,R2,R3,R4,R5):
  print(f"R={R}")  # Colab might (try to) take over again
  print(f"  reflexive    : {is_reflexive(R,Omega)}")
  print(f"  irreflexive  : {is_irreflexive(R,Omega)}")
  print(f"  symmetric    : {is_symmetric(R)}")
  print(f"  antisymmetric: {is_antisymmetric(R)}")
  print(f"  asymmetric   : {is_asymmetric(R)}")
  print(f"  transitive   : {is_transitive(R)}")
  print(f"  eq. relation : {is_reflexive(R,Omega) and is_symmetric(R) and is_transitive(R)}")
  print(f"  partial order: {is_reflexive(R,Omega) and is_antisymmetric(R) and is_transitive(R)}")
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

1. Basic concepts
   - Basic concepts of logic
   - Sets
   - Sequences, tuples, and words
   - The natural numbers, proof by induction
   - Relations
   - Functions
     - Basics
     - Injectivity, surjectivity and bjectivity

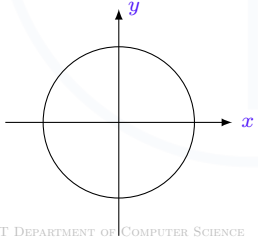
For personal use only, no commercial use – TUM CIT Department of Computer Science

**Def** A relation $R \subseteq A \times B$ is a (total) function/map(ping) **:iff** for every $a \in A$ there is exactly one $b \in B$ with $(a, b) \in R$.

- ▷ I.e.: for all $a \in A$ we have $|aR| = 1$ (reminder $aR := \{b \in B \mid (a, b) \in R\}$).
- ▷ $R$ is a partial function **:iff** $|aR| \leq 1$ for all $a \in A$.

**EG** Compare

$$\begin{aligned} R_1 &= \{(x, y) \in \mathbb{R}^2 \mid x^2 + y^2 = 1\} \\ R_2 &= \{(x, y) \in \mathbb{R}^2 \mid y \geq 0, x^2 + y^2 = 1\} \\ R_3 &= \{(x, y) \in [-1, 1] \times \mathbb{R} \mid y \geq 0, x^2 + y^2 = 1\} \end{aligned}$$




For personal use only, no commercial use – TUM CIT Department of Computer Science

**Def** A relation $R \subseteq A \times B$ is a (total) function/map(ping) **:iff** for every $a \in A$ there is exactly one $b \in B$ with $(a, b) \in R$.

- I.e.: for all $a \in A$ we have $|aR| = 1$ (reminder $aR := \{b \in B \mid (a, b) \in R\}$).
- $R$ is a partial function **:iff** $|aR| \leq 1$ for all $a \in A$.

- Conventions and related definitions:
  - "$f\colon A \to B$" short for "$f \subseteq A \times B$ is a function from $A$ to $B$". $A$ is then the domain, $B$ the codomain of $f$.
  - "$f(a)$" denotes the unique image (successor) of $a$ wrt. $f\colon A \to B$:

$$af = \{b \in B \mid (a, b) \in f\} \overset{|af|=1}{=} \{f(a)\}$$

    But $b \in B$ can have either no, one or multiple preimages (predecessors) $fb \in 2^A$.

⚠ "$afb$" **iff** "$b = f(a)$" ($a, b$ switch sides)

- Several ways of defining a function, e.g.:

$$f\colon \mathbb{R}\backslash\{0\} \to \mathbb{R}, x \mapsto x^{-1} \text{ or } f\colon \begin{cases} \mathbb{R} \setminus \{0\} \to \mathbb{R} \\ x \mapsto x^{-1} \end{cases} \text{ or } f\colon \mathbb{R}\backslash\{0\} \to \mathbb{R} \text{ with } f(x) := x^{-1}$$

For personal use only, no commercial use – TUM CIT Department of Computer Science

**Def** A relation $R \subseteq A \times B$ is a (total) function/map(ping) **:iff** for every $a \in A$ there is exactly one $b \in B$ with $(a, b) \in R$.

- ▷ I.e.: for all $a \in A$ we have $|aR| = 1$ (reminder $aR := \{b \in B \mid (a, b) \in R\}$).
- ▷ $R$ is a partial function **:iff** $|aR| \leq 1$ for all $a \in A$.

⚠ A function requires a domain $A$ and a codomain $B$.

- ▷ The different expressions $(1 - x)^{-1}$ and $\sum_{i=0}^{\infty} x^i$ describe the same function from $(-1, 1)$ to $\mathbb{R}$ but not from $\mathbb{R}_{>1}$ to $\mathbb{R} \cup \{\infty\}$.

  Similarly, different "code" can describe the same "function" ("input-output relation"; "**syn**").
- ▷ A partial function $f \subseteq A \times B$ can be extended to a (total) function

$$f_\perp := f \cup \{(a, \perp) \mid a \in A, af = \emptyset\} \subseteq A \times (B \cup \{\perp\})$$

  where $\perp \notin B$ is a so-far unused symbol/value which can be read as "undefined" or "error" or "exception".

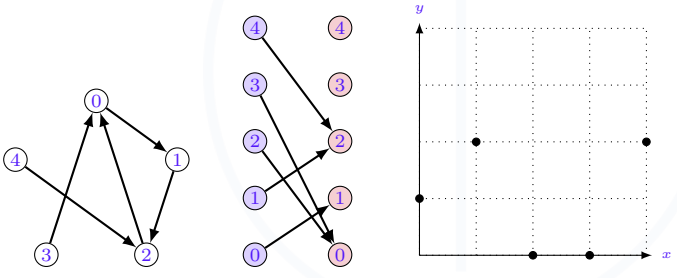
⚠ Although a relation $R \subseteq A \times B$ is in general not a function $A \to B$, it always corresponds uniquely to a function $2^A \to 2^B, X \mapsto XR$ (cf. "determinization").

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Graphical representations of functions

- Possible graphical representations for a function $f\colon A \to B$:
  1. As a relation/graph: one node for every $x \in A \cup B$.
  2. As a bipartite graph: $A$ (left) and $B$ (right) (potentially two nodes per object).
  3. As points in $\mathbb{R}^{n+m}$ if $A \subseteq \mathbb{R}^n$ and $B \subseteq \mathbb{R}^m$ (e.g. $\vec{x}(t), h(\vec{x})$).

EG $f\colon \mathbb{Z}_5 \to \mathbb{Z}_5, x \mapsto (x^2+1) \bmod 5$ (remainder after dividing by 5)



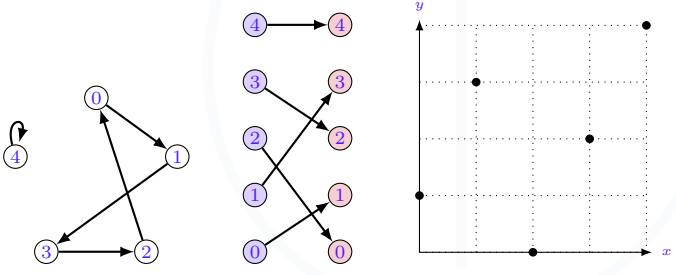

(Wrt. 1 : as $A$ is finite and $f$ a function, we must end in a cycle; cf. $R^* = R^{<|A|}$.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Graphical representations of functions

- Possible graphical representations for a function $f\colon A \to B$:
  1. As a relation/graph: one node for every $x \in A \cup B$.
  2. As a bipartite graph: $A$ (left) and $B$ (right) (potentially two nodes per object).
  3. As points in $\mathbb{R}^{n+m}$ if $A \subseteq \mathbb{R}^n$ and $B \subseteq \mathbb{R}^m$ (e.g. $\vec{x}(t), h(\vec{x})$).

EG $f\colon \mathbb{Z}_5 \to \mathbb{Z}_5, x \mapsto (2x+1) \bmod 5$ (remainder after dividing by 5)



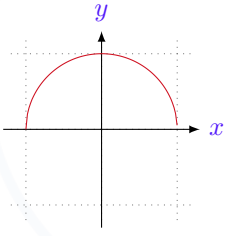

(Wrt. 1 : as $f$ is bijective, only cycles; see cycle notation later.)

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Graphical representations of functions

- Possible graphical representations for a function $f\colon A \to B$:
  1. As a relation/graph: one node for every $x \in A \cup B$.
  2. As a bipartite graph: $A$ (left) and $B$ (right) (potentially two nodes per object).
  3. As points in $\mathbb{R}^{n+m}$ if $A \subseteq \mathbb{R}^n$ and $B \subseteq \mathbb{R}^m$ (e.g. $\vec{x}(t), h(\vec{x})$).

EG $f = \{(x, y) \in [-1, 1] \times \mathbb{R} \mid x^2 + y^2 = 1, y \geq 0\}$

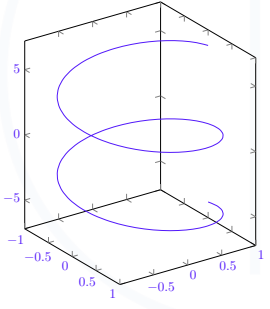



For personal use only, no commercial use – TUM CIT Department of Computer Science

# Graphical representations of functions

- Possible graphical representations for a function $f\colon A \to B$:
  1. As a relation/graph: one node for every $x \in A \cup B$.
  2. As a bipartite graph: $A$ (left) and $B$ (right) (potentially two nodes per object).
  3. As points in $\mathbb{R}^{n+m}$ if $A \subseteq \mathbb{R}^n$ and $B \subseteq \mathbb{R}^m$ (e.g. $\vec{x}(t), h(\vec{x})$).

EG $f\colon \mathbb{R} \to \mathbb{R}^2, t \mapsto (\sin(\frac{2\pi}{6}t), \cos(\frac{2\pi}{6}t))$, $f \subseteq \mathbb{R} \times \mathbb{R}^2 \cong \mathbb{R}^3$ ($t$: vertical axis)

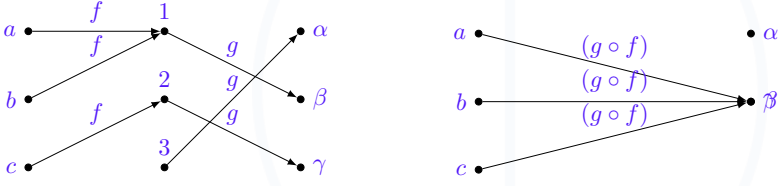



For personal use only, no commercial use – TUM CIT Department of Computer Science

**Def** Function composition:

For $f\colon A \to B$ and $g\colon B \to C$ define $(g \circ f)$ ("$g$ after $f$") by

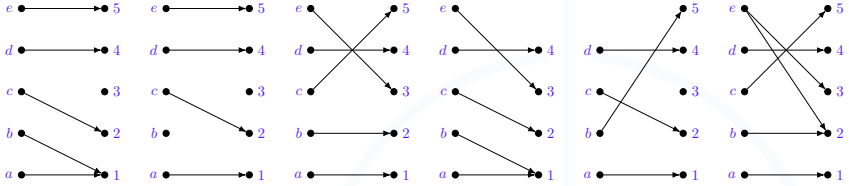
$$(g \circ f)\colon A \to C, a \mapsto g(f(a))$$




⚠ The composition of functions is associative but not commutative.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Injective, surjective and bijective functions




**Def** A function $f\colon A \to B$ is

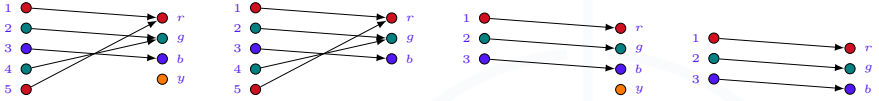
- ▷ injective **:iff** $|fb| \leq 1$ for every $b \in B$ (at most one preimage/predecessor).
  i.e. for all $a, a' \in A$: if $f(a) = f(a')$, then $a = a'$.
- ▷ surjective **:iff** $|fb| \geq 1$ for every $b \in B$ (at least one preimage/predecessor).
  i.e. for every $b \in B$ there is some $a \in A$ with $f(a) = b$.
- ▷ bijective **:iff** $|fb| = 1$ for all $b \in B$ (exactly one preimage/predecessor).
  i.e. both injective and surjective.

**Fct** Let $f\colon A \to B$ and $g\colon B \to C$.

- ▷ If $f, g$ are both injective (surjective), then is $(g \circ f)$ injective (surjective).
- ▷ If $(g \circ f)$ is injective/surjective, then $f$ is injective/$g$ is surjective.

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Inverse function




**Fct** Let $f\colon A \to B$ with $f^\top = \{(b,a) \mid (a,b) \in f\}$.

- $f$ is injective **iff** $|bf^\top| \leq 1$ for all $b \in B$.
- $f$ is surjective **iff** $|bf^\top| \geq 1$ for all $b \in B$.
- Thus: $f$ is bijective **iff** $f^\top$ is a function from $B$ to $A$.

**Def** If $f\colon A \xrightarrow{\text{bij}} B$, then $f^{-1} := f^\top$ is called the inverse (function) of $f$.

**Cor** If $f$ is bijective, then $f^{-1}$ is bijective as $(f^{-1})^\top = (f^\top)^\top = f$.
In particular: $f = (f^{-1})^{-1}$ and $f^{-1} \circ f = \mathrm{id}_A$ and $f \circ f^{-1} = \mathrm{id}_B$.

**Fct** If $f\colon A \to A$ injective or surjective and $A$ finite, then $f$ is bijective.

⚠ The notation "$f^{-1}$" is usually overloaded: (see images and preimages)

**Def** If $f$ is not bijective: $f^{-1}\colon B \to 2^A$ with $f^{-1}(b) := fb = \{a \in A \mid f(a) = b\}$.
And in general: $f^{-1}\colon 2^B \to 2^A$ with $f(Y) := \bigcup\{fb \mid b \in Y\}$.

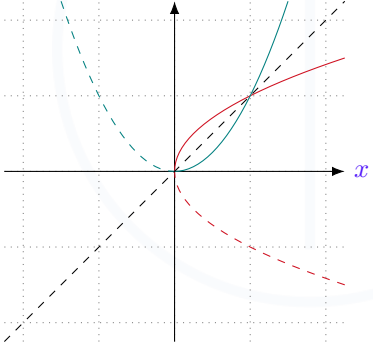
For personal use only, no commercial use – TUM CIT Department of Computer Science

# Inverse function: reflection, monotonicity

**Fct** For $f\colon A \to B$ with $(A, \leq_A)$ and $(B, \leq_B)$ totally ordered and for all $a <_A a'$ always $f(a) <_B f(a')$, then $f$ is injective and $f\colon A \to f(A)$ is bijective.

⚠ For $f\colon \mathbb{R} \to \mathbb{R}$, we can visualize $f$ as points $(x, f(x))$ in the plane; then $f^\top = \{(f(x), x) \mid x \in \mathbb{R}\}$ is the reflection wrt. $\{(x, x) \mid x \in \mathbb{R}\}$.

**EG** $f\colon \mathbb{R} \to \mathbb{R}, x \mapsto x^2$ is not bijective, so $f^\top$ is not a function.
But $g\colon \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}, x \mapsto x^2$ is bijective with $g^{-1}\colon \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}, x \mapsto \sqrt{x}$.



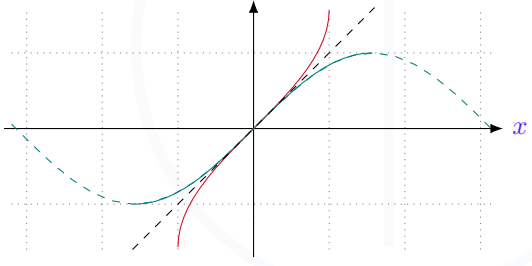

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Inverse function: reflection, monotonicity

**Fct** For $f\colon A \to B$ with $(A, \leq_A)$ and $(B, \leq_B)$ totally ordered and for all $a <_A a'$ always $f(a) <_B f(a')$, then $f$ is injective and $f\colon A \to f(A)$ is bijective.

⚠ For $f\colon \mathbb{R} \to \mathbb{R}$, we can visualize $f$ as points $(x, f(x))$ in the plane; then $f^\top = \{(f(x), x) \mid x \in \mathbb{R}\}$ is the reflection wrt. $\{(x, x) \mid x \in \mathbb{R}\}$.

**EG** $f\colon \mathbb{R} \to \mathbb{R}, x \mapsto \sin(x)$ is not bijective, so $f^\top$ is not a function.
But $g\colon [-\frac{\pi}{2}, \frac{\pi}{2}] \to [-1, 1], x \mapsto \sin(x)$ is bijective with $g^{-1} = \arcsin$.




For personal use only, no commercial use – TUM CIT Department of Computer Science

# Functions (set theory) in Python

```
import networkx as nx
import matplotlib.pyplot as plt
from typing import *

f_def = lambda x: x + 1       # just x ↦ x + 1, not yet a function in the strict sense
g_def = lambda x: (x+1) % 5   # (see lambda calculus/terms)

domain = {x for x in range(10)}
# domain = {x/2 for x in range(20)}  # for comparison
f = {(x,f_def(x)) for x in domain}
g = {(x,g_def(x)) for x in domain}

def plot_func(f: set[tuple[Any,Any]]):
  print(f)
  f = sorted(f)  # arrange set into sequence sorted w.r.t. preimage
  # f as table/two-line notation
  print(f"Dom(f)={set(x for (x,y) in f)}")
  print(f"Im(f)={set(y for (x,y) in f)}")
  G = nx.DiGraph()
  G.add_edges_from(f)
  nx.draw(G, with_labels=True, pos=nx.kamada_kawai_layout(G, pos=nx.spring_layout(G, seed=42)))
  plt.show()
  plt.plot([x for (x,y) in f], [y for (x,y) in f], 'x')
  plt.plot([x for (x,y) in f], [y for (x,y) in f], ':')
  # TODO: plot also transposed relation
  plt.show()

plot_func(f)
plot_func(g)
```

For personal use only, no commercial use – TUM CIT Department of Computer Science

# Functions (set theory) in Python as `dict`



```python
import networkx as nx
import matplotlib.pyplot as plt
from typing import *

# A Python `dict` represents a function with finite domain
# (C++ and Java use `map` instead of `dict`.)
f = {x: x+1 for x in range(10)}       # Python uses `:` for ↦
g = {x: (x+1) % 5 for x in range(10)}


def plot_func(f: dict[Any,Any]):
  print(f)
  dom_f = sorted(f.keys())              # `keys()` returns the domain of `f`, `values()` returns its image
  # f as table/two-line notation
  print(f"Dom(f)={set(x for x in dom_f)}")
  print(f"Im(f)={set(f[x] for x in dom_f)}")   # `f[x]` is the image of `x` w.r.t. `f` (note square brackets)
  G = nx.DiGraph()
  G.add_edges_from(f.items())           # `f.items()` is the actual relation
  nx.draw(G, with_labels=True, pos=nx.kamada_kawai_layout(G, pos=nx.spring_layout(G, seed=42)))
  plt.show()
  plt.plot([x for x in dom_f], [f[x] for x in dom_f], 'x')
  plt.plot([x for x in dom_f], [f[x] for x in dom_f], ':')
  plt.show()

plot_func(f)
plot_func(g)
```

For personal use only, no commercial use – TUM CIT Department of Computer Science